İşletim Sistemleri
(Operating Systems)

Bellek Yönetimi
(Memory Management)

Modern Bellek Yönetimi Teknikleri

- Bitişik bellek ayırma (Contiguous memory allocation) yöntemlerinden olan hem değişmez (fixed) hem de dinamik (dynamic) bölümleme, belleği verimsiz kullanırlar: biri iç parçalanmaya diğeri ise dış parçalanmaya neden olurlar.
- Belleğe yeni process'lerin yüklenmesi ve sonlanan process'lerin bellekten kaldırılması sırasında, bellekte verimsiz boşluklar oluşur.
- Belleğin çeşitli yerlerinde küçük parçaların kalması sonucunda bellekte toplamda yeterli yer olmasına karşılık yeni bir process bu küçük parçalara bölünemeyeceği için bellekte yetersiz yer problemi ile karşılaşılacaktır.
- İç ve dış parçalanma sorununun üstesinden gelmek için Bitişik olmayan bellek ayırma (Non-Contiguous memory allocation) yöntemlerinden olan **sayfalı bellek yönetimi (paging)** ve **kesimli bellek yönetimi (segmentation)** kullanılabilir.

Modern Bellek Yönetimi Teknikleri

Bitişik olmayan bellek ayırma (Non-Contiguous memory allocation) yöntemleri;

Sayfalı Bellek Yönetimi (Paging),

Kesimli Bellek Yönetimi (Segmentation)

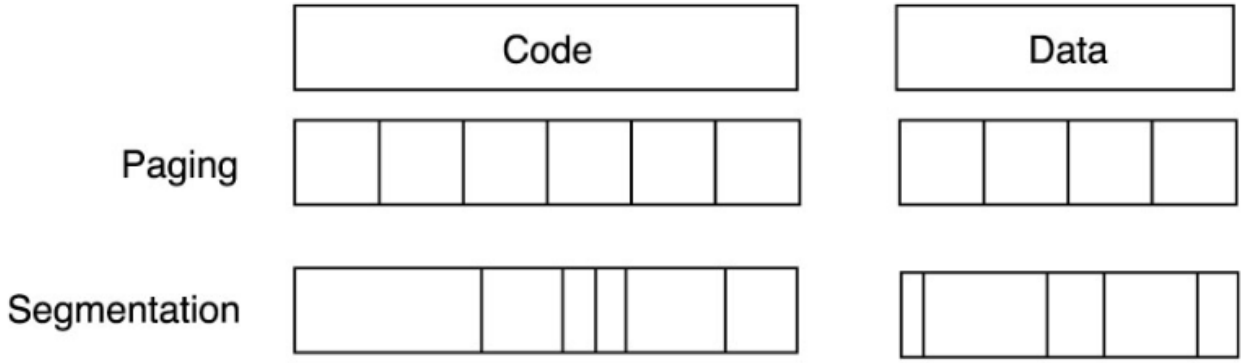
ve bu ikisinin karması

Kesimli-Sayfalı Bellek Yönetimi (Segmentation - Paging)

Modern Bellek Yönetimi Teknikleri

- Sayfalı Bellek Yönetimi (Paging),
- Bellek, sabit boyutlu çerçevelere (frame) bölünür.
- Her çerçeve bir taban fiziksel adrese (base physical address) sahiptir.

- Kesimli Bellek Yönetimi (Segmentation),
- Bellek değişken uzunlukta segmentlere bölünür.
- Her segment bir taban fiziksel adres (base physical address) + uzunluğa (length) sahiptir.



Sayfalı Bellek Yönetimi (Paging)

- Sayfalama (paging) yönteminde,
- Fiziksel ana bellek (RAM) adres uzayı, birbirini izleyen, eşit uzunlukta bloklara bölünür. Bu bloklar **çerçeve (frame)** olarak adlandırılır.
- Process mantıksal adres uzayı da, birbirini izleyen, eşit uzunlukta bloklara bölünür. Bu bloklar **sayfa (page)** olarak adlandırılır.

- Çerçeve (Frame) = Fiziksel bellek bloğu
- Sayfa (Page) = Mantıksal bellek bloğu

Sayfalı Bellek Yönetimi (Paging)

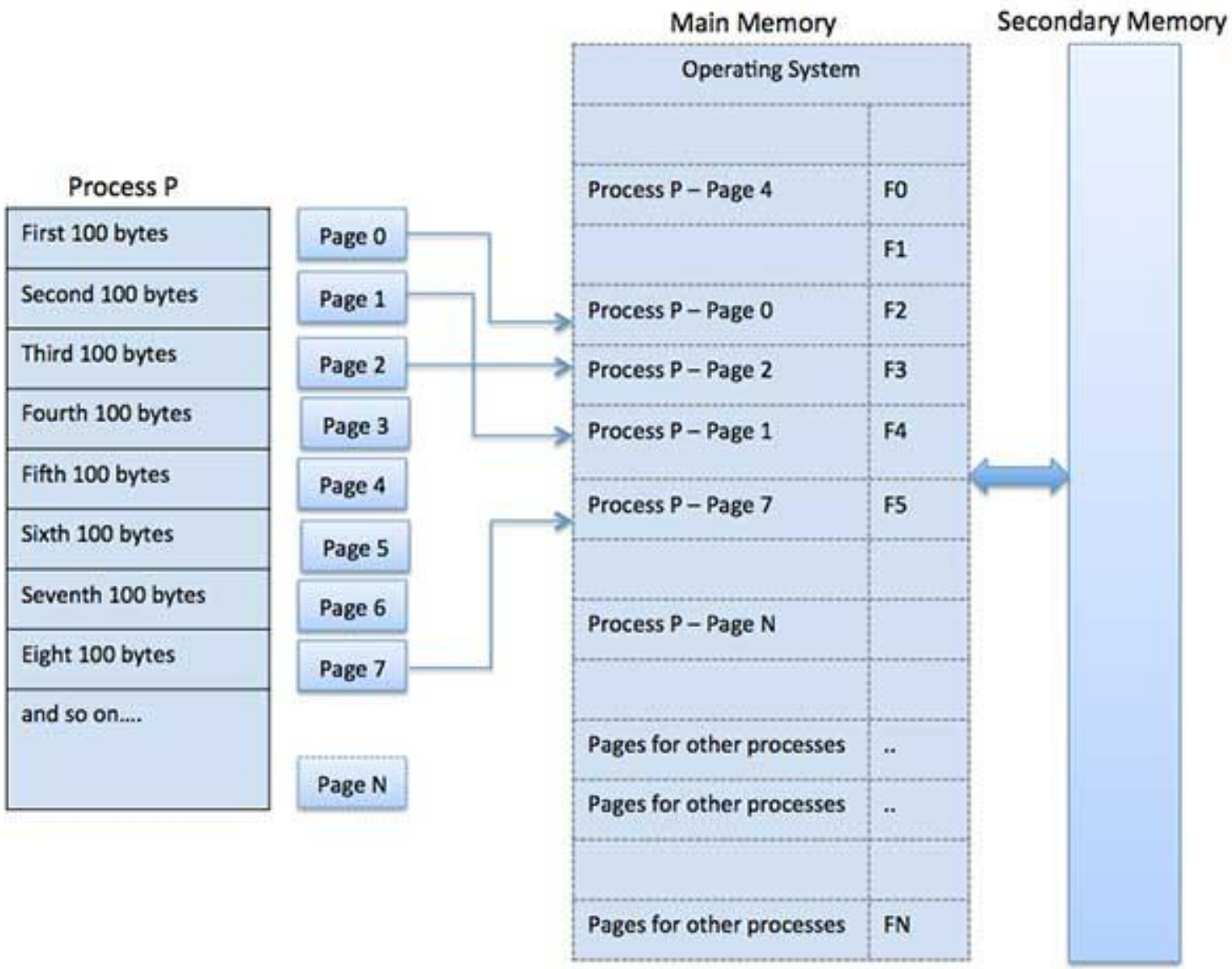
❖ Eşit boylu bellek blokları: çerçeve (frame)

❖ Eşit boylu process blokları: sayfa (page)

- Genelde fiziksel bellek çerçeve uzunlukları (frame size) ve process sayfa uzunlukları (page size) birbirine eşittir.

- Sayfa boyutu donanım tarafından tanımlanır. Sayfa boyu, 512 bayt ve 16 MB arasında olabilir, genellikle de 4-8 KB arasında tanımlanır.

Sayfalı Bellek Yönetimi (Paging)



Sayfalı Bellek Yönetimi (Paging)

- Her process eşit uzunluktaki sayfalara, ana bellek de eşit uzunluktaki çerçevelere ayrılmıştır. (Page Size = Frame Size)
- **İşletim sistemi her process için bir sayfa tablosu (page table) tutar.**
- Sayfa Tablosu, sayfa numarasının çerçeve numarasına eşlenmesini gerçekleştiren bir veri yapısıdır.
- Sayfa tabloları bellekte process kontrol bloğunun (PCB) bir parçası olarak ana bellekte tutulur.
- Bir bağlam anahtarlama (context switching) yapılırken geçerli sayfa tablosu kaydedilir ve geri yüklenir.
- **Process'lerin hangi sayfasının fiziksel bellekte hangi çerçevede olduğu bilgisi process'lerin sayfa tablosunda tutulmaktadır.**

Sayfalı Bellek Yönetimi (Paging)

- Yandaki örnekte;

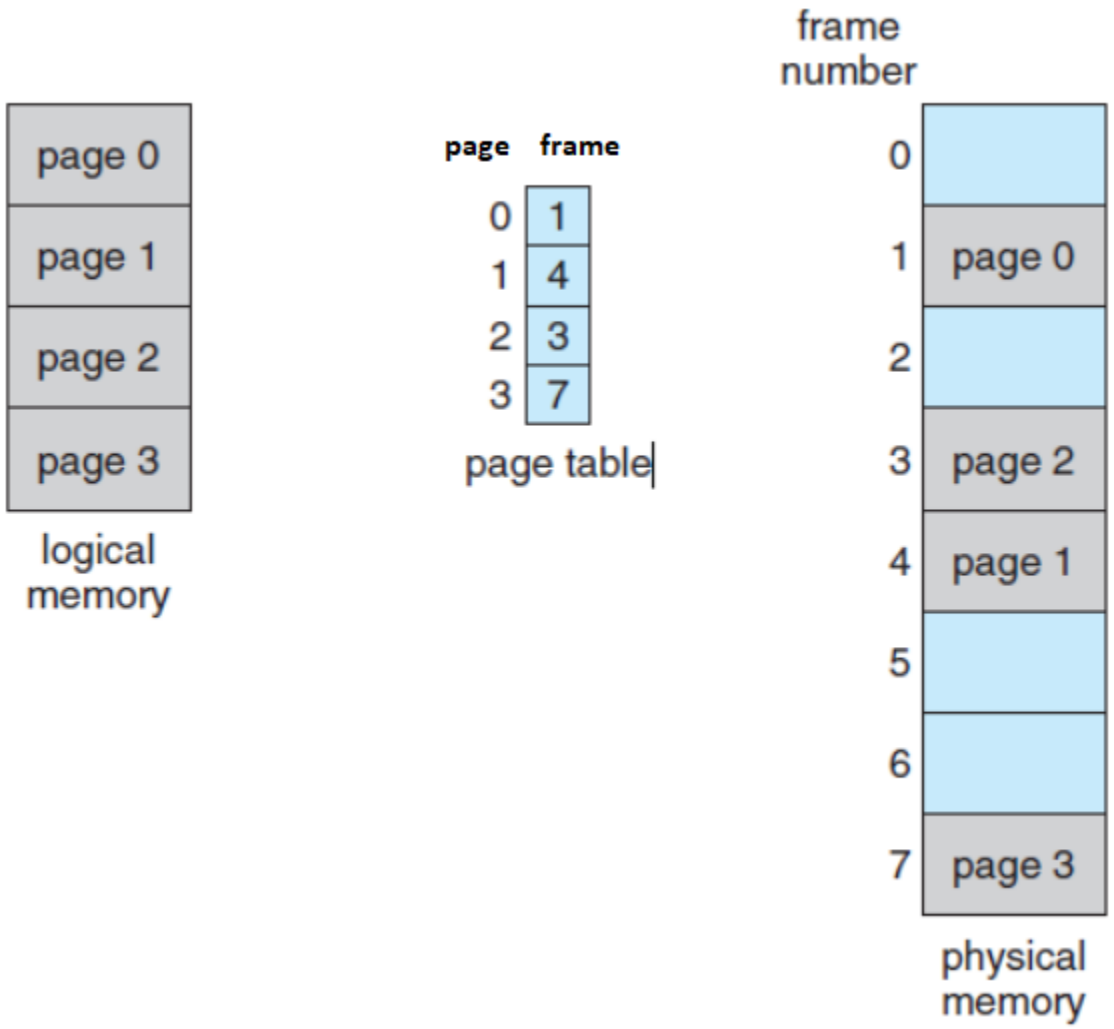
sayfa 0, çerçeve 1'de,

sayfa 1 çerçeve 4'te,

sayfa 2 çerçeve 3'te

sayfa 3 çerçeve 7'de

yer almaktadır.



Sayfalı Bellek Yönetimi (Paging)

P1 PMT

Page	Frame
0	5
1	12
2	15
3	7
4	22

P2 PMT

Page	Frame
0	10
1	18
2	1
3	11

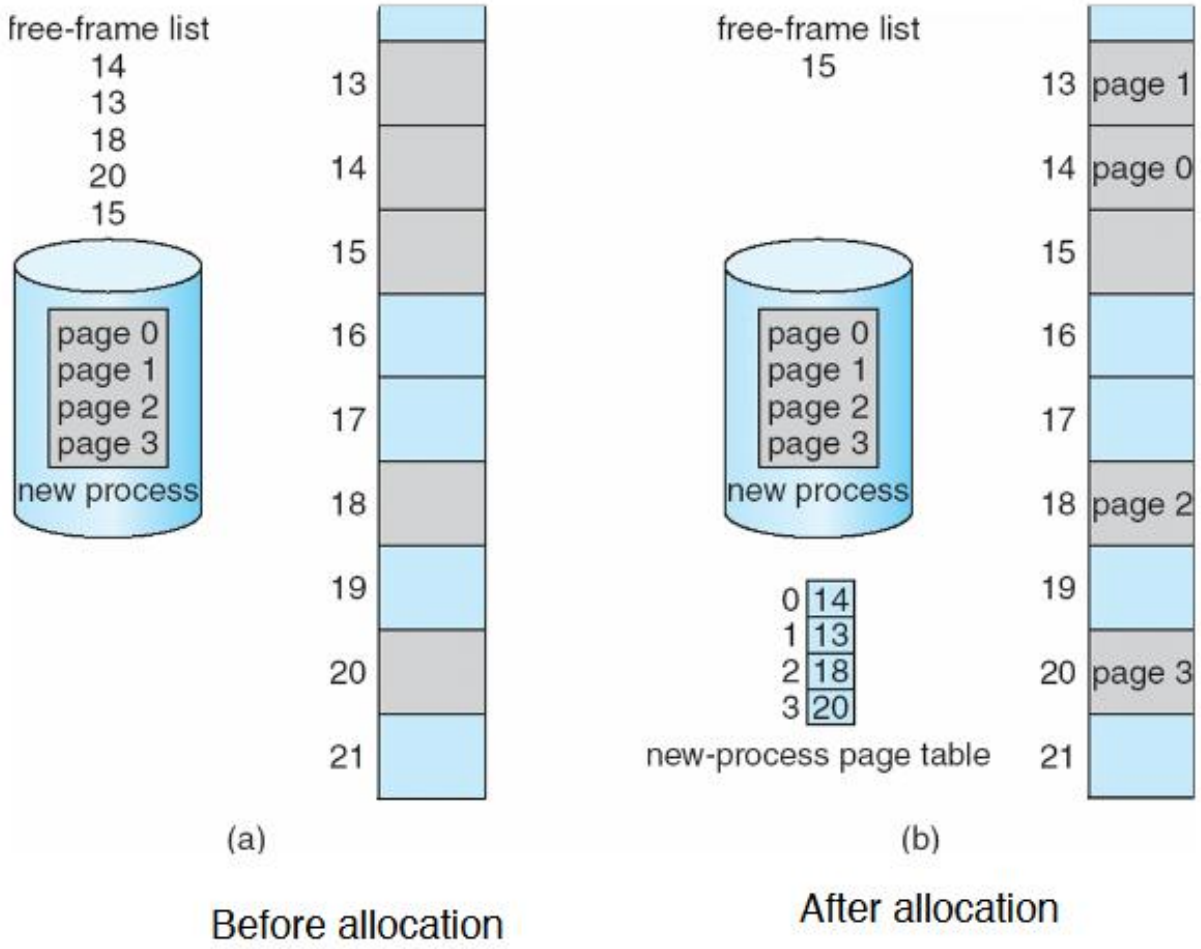
Memory

Frame	Contents
0	
1	P2/Page2
2	
3	
4	
5	P1/Page0
6	
7	P1/Page3
8	
9	
10	P2/Page0
11	P2/Page3
12	P1/Page1
13	
14	
15	P1/Page2

.
.
.

Sayfalı Bellek Yönetimi (Paging)

- İşletim sistemi bu yöntemi kullanırken boş çerçevelerin de listesini (free frame list) tutar.



Sayfa tablosu girdileri (Page Table Entries)

- **Modify bit (M) :** Sayfa üzerinde yazma yapılıp yapılmadığını belirtir. Sayfaya bir yazma yapıldığında M biti 1 olur.
- **Reference bit (R) :** Sayfaya erişilip erişilmediğini belirtir. Sayfaya erişildiğinde yani bir okuma veya yazma gerçekleştiğinde R biti 1 olur.
- **Valid bit (V) :** Durum belirteci olarak bilinir. Sayfa tablosunda, ilgili sayfanın bellekte yüklü olup olmadığını gösterir . Yani hangi sayfaların bellekte hangilerinin ikincil bellek (disk) üzerinde olduğunu ayırt etmek için kullanılır. valid:1, invalid:0
- **Protection bits (Prot) :** Sayfada hangi işlemlere izin verildiğini belirtir. Oku (read), yaz (write), yürüt (execute).
- **Page frame number :** Fiziksel bellekteki çerçeve (frame) numarasını gösterir.

1	1	1	2	20
M	R	V	Prot	Page Frame Number

- Bu bitler her bellek başvurusunda güncellenmelidir, bu yüzden donanım ile yapılmalıdır.

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

- Sayfalı bellek yönetimi olan sistemlerde CPU tarafından oluşturulan her adres (mantıksal adres) iki bölümden oluşmaktadır: sayfa numarası ve sayfa ofseti.

  - **Sayfa numarası (p) :** Mantıksal adresin bulunduğu sayfanın numarasını gösterir.
  - **Sayfa ofseti (d) :** sayfa taban adresi ile birleştirilerek sayfa içindeki adres konumunun belirlenmesinde kullanılır.

  - **Sayfa ofseti sayfa boyundan büyük olamaz.**

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

Mantıksal Adres ⇒ Sayfa numarası (p) + Sayfa ofseti (d)
Logical Address ⇒ Page number (p) + Page offset (d)

Mantıksal adres uzayı ve sayfa boyu 2'nin katları şeklindedir.

Mantıksal adres uzayı 2^m bayt ve sayfa boyu 2^n bayt olduğunda;

page number	page offset
p	d
m -n	n

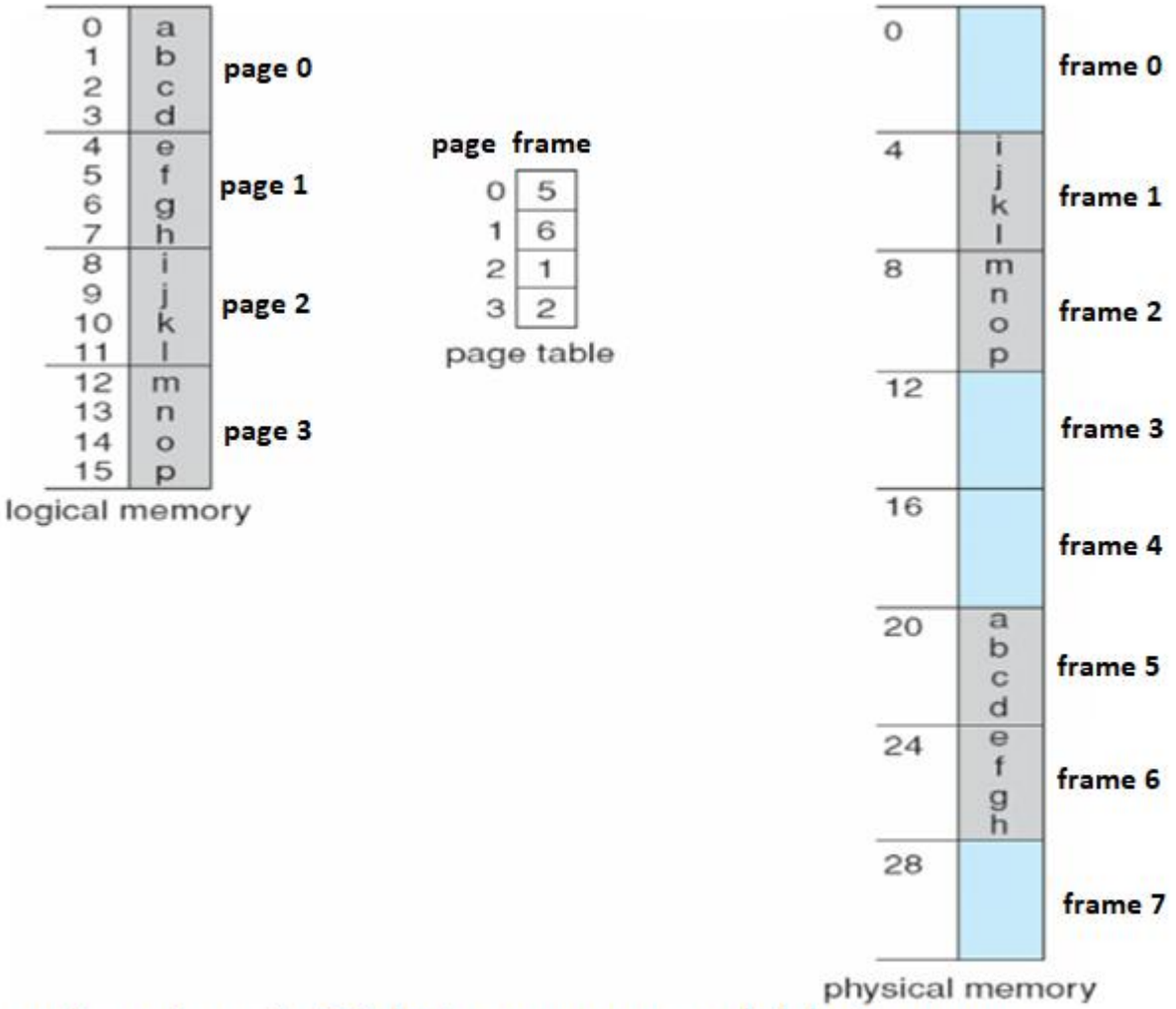
Mantıksal adres uzayı : 2^m (bayt) **Sayfa boyu : 2^n (bayt)**
Sayfa numarası (p) = m-n (bit)
Sayfa ofseti (d) = n (bit)

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

- 65536 (2^{16}) bayt olarak verilen 16 bit uzunluğunda bir mantıksal adres
- ve
- 1024 (2^{10}) bayt sayfa boyu için,

- (Burada m=16 ve n=10)
- p= m-n= 16-10= 6 bit d= n= 10 bit
- Mantıksal adres 16 bit uzunluğundadır ve yüksek anlamlı 6-bit sayfa numarasını, düşük anlamlı 10-bit ise sayfa ofset değerini gösterir.

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler



n=2 and *m*=4 32-byte memory and 4-byte pages

Mantıksal adres uzayı : 16 bayt = 2^4 bayt

Sayfa uzunluğu : 4 bayt = 2^2 bayt

Mantıksal adres uzayı 2^m ve sayfa boyu 2^n

(Burada m=4 ve n=2)

Mantıksal adres uzayında : 16/4 = 4 sayfa (page) bulunuyor.

Fiziksel bellek uzayı : 32 bayt = (2^5) bayt

Fiziksel bellek uzayında : 32/4 = 8 çerçeve (frame) bulunuyor.

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

- Fiziksel bellek adresleri bir **çerçeve numarası (frame number) ve çerçeve ofseti (frame offset)** ile temsil edilir.
  - **Çerçeve numarası (frame number) (f) :** Fiziksel adresin bulunduğu çerçevenin numarasını gösterir.
  - **Çerçeve ofseti (frame offset) (d) :** Çerçeve taban adresi ile birleştirilerek çerçeve içindeki adres konumunun belirlenmesinde kullanılır.
  - **Sayfa ofseti (page offset) (d) = çerçeve ofseti (frame offset) (d)**

Frame number	Offset
f	d

Fiziksel Adres ⇒ Çerçeve numarası (f) + çerçeve ofseti (d)

Physical Address ⇒ Frame number (f) + frame offset (d)

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

- Fiziksel Adres : 12 bit, Fiziksel Adres Uzayı = 4K = 4096 bayt =2^{12} bayt
- Mantıksal Adres : 13 bit, Mantıksal Adres Uzayı = 8K =8192 bayt =2^{13} bayt
- Sayfa boyutu = çerçeve boyutu = 1K = 1024 bayt = 2^{10} bayt olduğu kabul edilirse;
-
- Çerçeve sayısı = Fiziksel adres uzayı / çerçeve boyutu = 4K / 1K = 4
- Sayfa sayısı = mantıksal adres uzayı / sayfa boyutu = 8K / 1K = 8
- Mantıksal adres (Logical address)

13 bit	
3 bit	10 bit
p	d

- Fiziksel adres (Physical address)

12 bit	
2 bit	10 bit
f	d

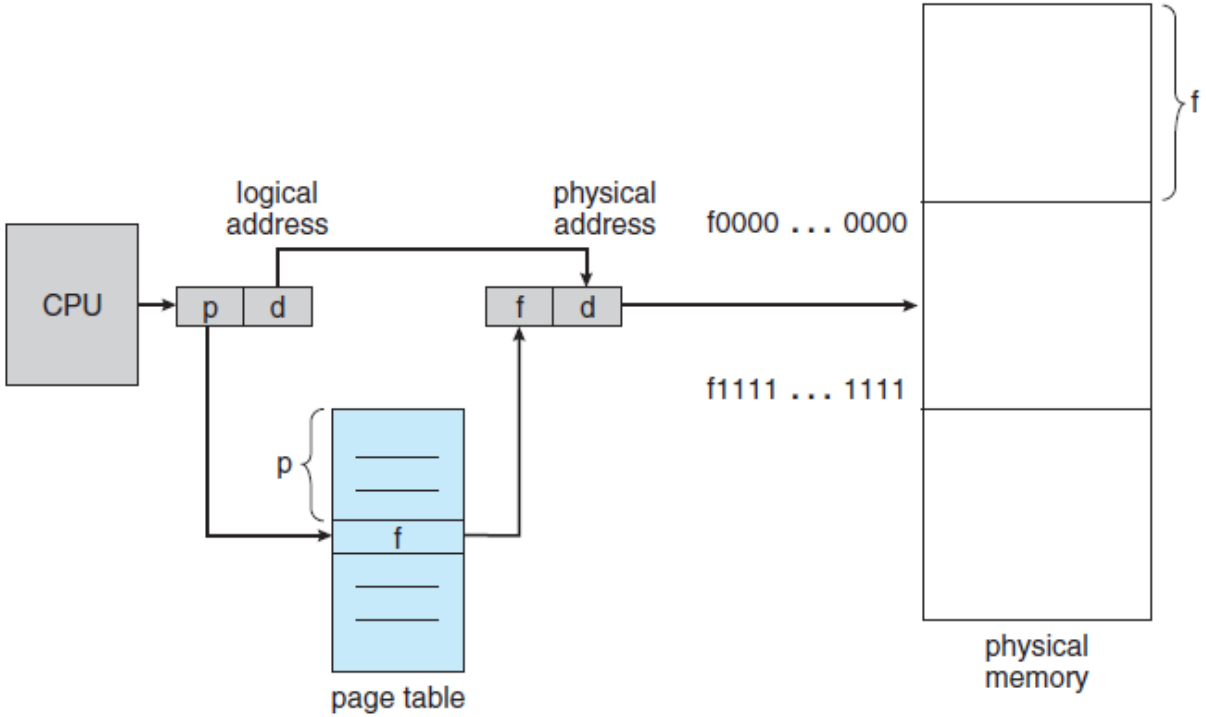
Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler

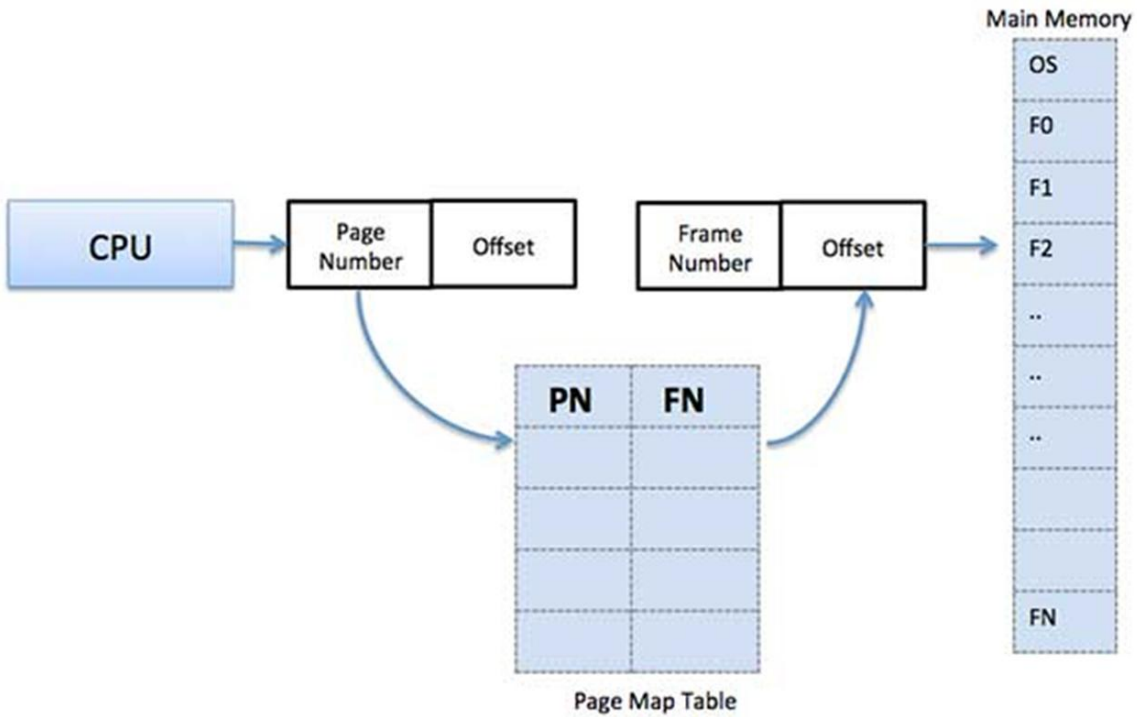
- Mantıksal adresin fiziksel bellekteki karşılığının bulunması için sayfa tablosundan faydalanılır.

- Sayfa tablosunda, mantıksal adres uzayının hangi sayfasının fiziksel bellekte hangi çerçevede olduğu bilgisi tutulmaktadır.

- Sayfa ve çerçeve boylarının ikinin kuvvetleri şeklinde seçilmesi dönüşüm hesaplamalarını basitleştirir.

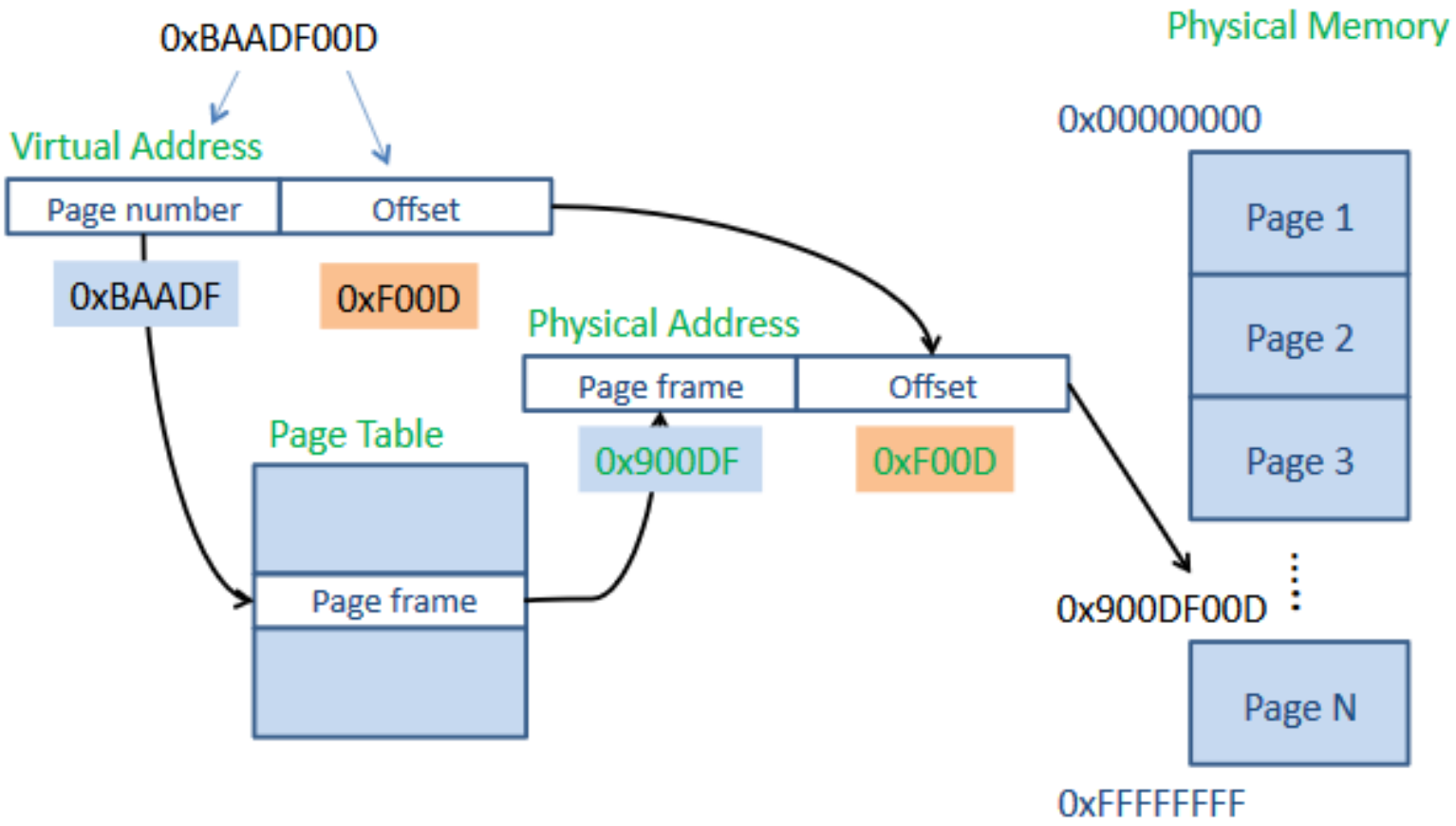
Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler





Sayfalar, CPU tarafından oluşturulan Mantıksal Adres ile temsil edilir. Bu Sayfalar, MMU tarafından Fiziksel Adres ile eşleştirilir. Bu nedenle, bu haritalamaya yardımcı olması için Sayfa Tablosu kullanılır. Mantıksal bir adresi, Fiziksel bir adrese eşleştirmek için, önce sayfa tablosundaki belirtilen sayfadaki çerçeve numarası bulunur. Ardından, çerçeve numarası ofset ile toplanır.

Sayfalı Bellek Yönetimi
Mantıksal-Fiziksel adresler



Sayfalı Bellek Yönetimi

Mantıksal Adresten Fiziksel Adrese Eşleştirme :

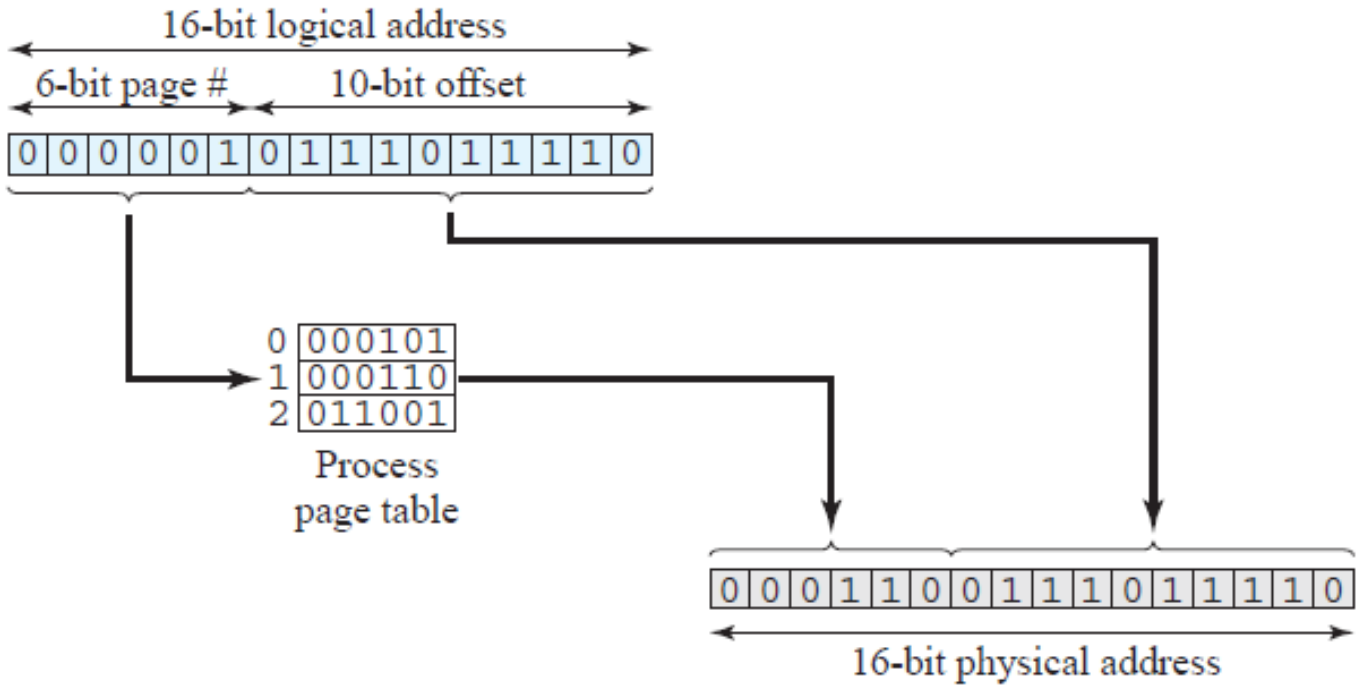
Mantıksal adres 16 bit uzunluğunda ve p= 6 bit ve d= 10 bit olsun.

Mantıksal adres 0000010111011110

sayfa numarası 1, ofset 478

Sayfa tablosunda, sayfa 1 içinde ikili 000110 (ondalık 6) değerinin bulunduğunu varsayalım.

Bu değer fiziksel adres uzayındaki çerçeve numarasını gösterir.



Fiziksel adres ⇒ çerçeve numarası + ofset = 6 + 478 = 0001100111011110

Örnek-1 :

- 4 bayt uzunluğunda sayfalardan oluşan 16 bayt mantıksal adres uzayı ve 32 bayt boyutunda bir fiziksel bellek bulunduğunu varsayınız. Sayfa tablosu yanda verilmiştir.
- Buna göre aşağıdaki mantıksal adresler için fiziksel adresleri eşleştiriniz.

a) 0

b) 3

c) 4

d) 13

0	5
1	6
2	1
3	2

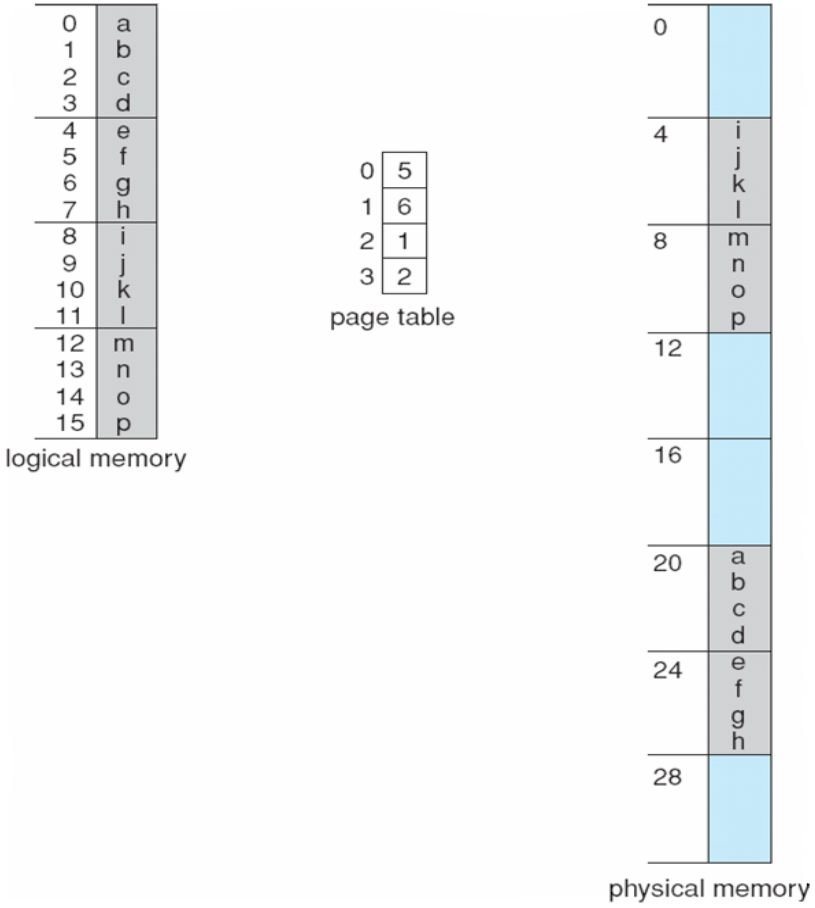
Sayfa tablosu

Örnek-1 :

- **Çözüm :**
- Sayfa boyu : 4 bayt =2^2 bayt,
- Mantıksal adres uzayı : 16 bayt = 2^4 bayt
- Mantıksal bellekte 16/4 = 4 sayfa (page) bulunuyor.

- Çerçeve boyu : 4 bayt =2^2 bayt,
- Fiziksel bellek uzayı : 32 bayt = 2^5 bayt
- Fiziksel bellekte 32/4 = 8 çerçeve (frame) bulunuyor.

- Her bir mantıksal adresin sayfa numarası ve sayfa ofsetinin bulunması gerekir.
- Sayfa numarası da, sayfa tablosundan çerçeve numarasını bulmak için kullanılacaktır.



Örnek-1 :

Mantıksal adres uzayı 2^m ve sayfa boyu 2^n

Mantıksal adres uzayı 16 bayt = 2^4 bayt

Sayfa uzunluğu 4 bayt = 2^2 bayt

(Burada m=4 ve n=2)

Mantıksal adres = p + d

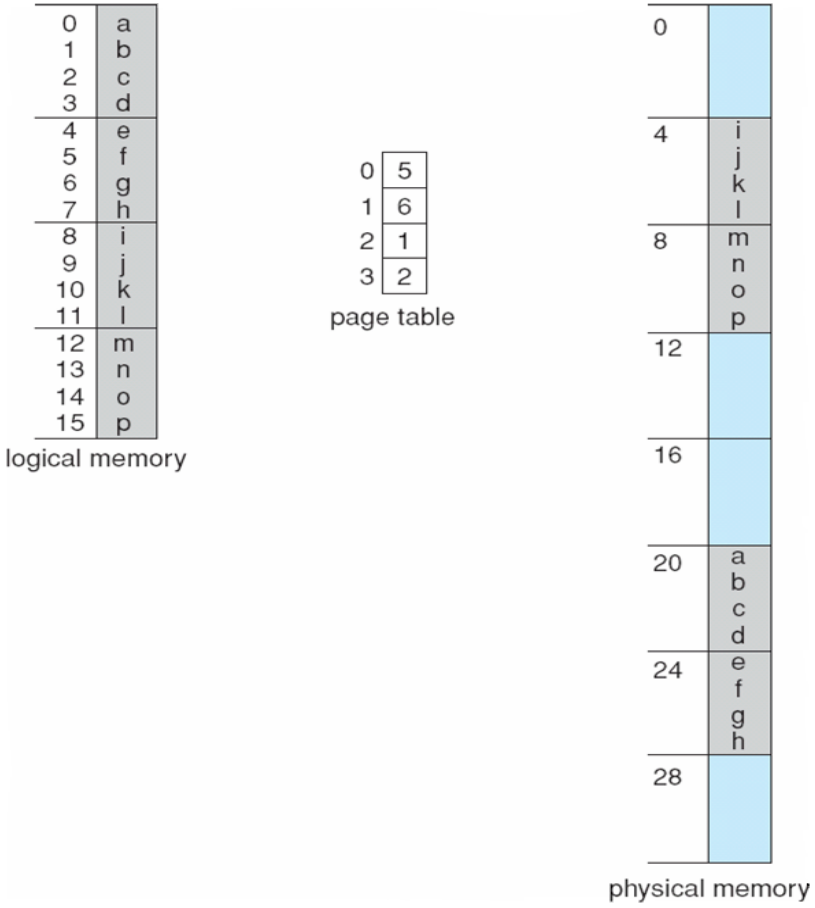
p= m-n= 4-2= 2 bit d= n= 2 bit

Mantıksal adres 4 bit uzunluğundadır.

2 bit sayfa numarası ve 2 bit sayfa ofsetidir.

Fiziksel adres 5 bit uzunluğundadır.

3 bit çerçeve numarası ve 2 bit sayfa ofsetidir.



Örnek-1:

a)

mantıksal adres : 0 = 0000

sayfa numarası = 0

sayfa ofseti = 0

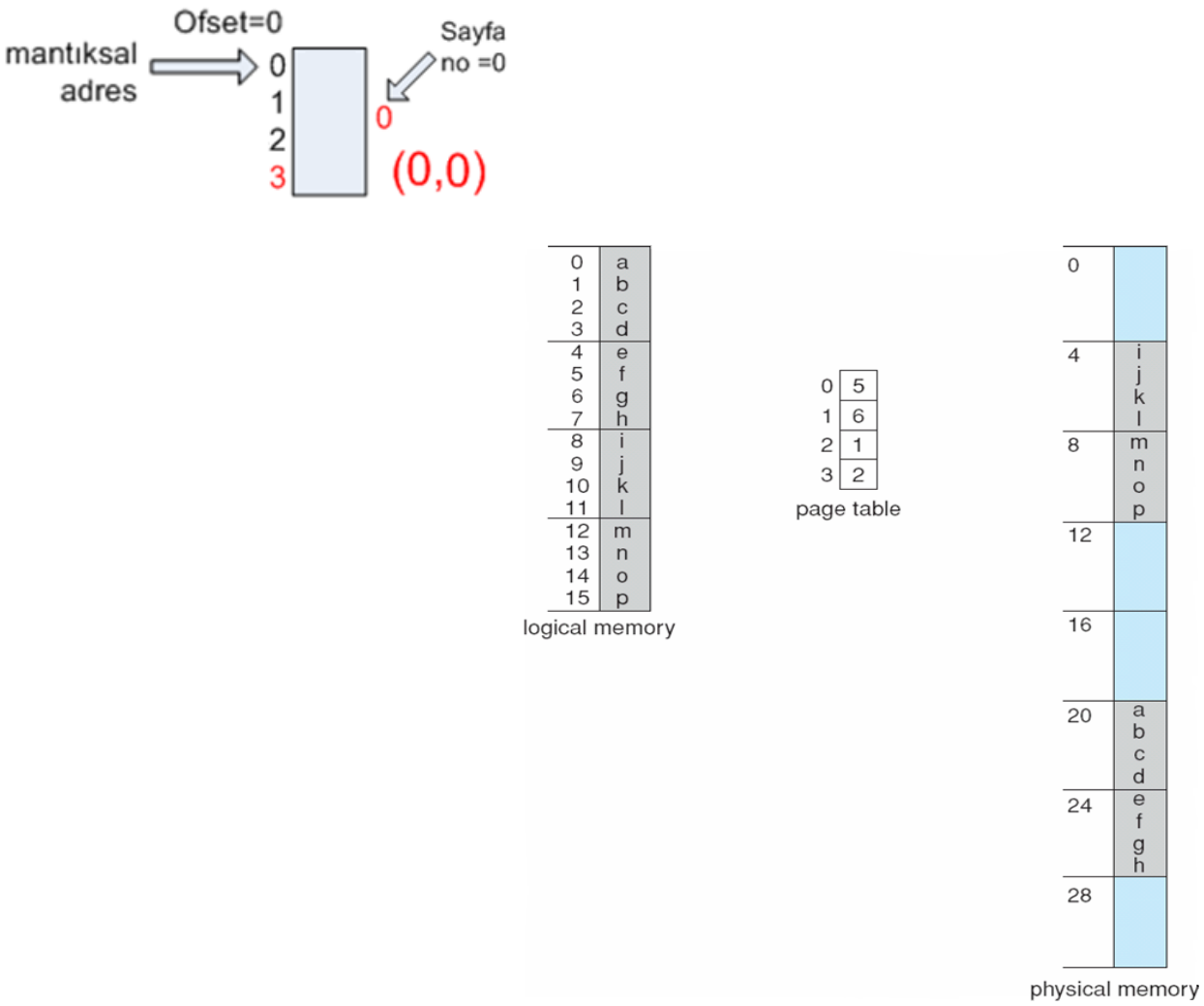
0 numaralı sayfanın bulunduğu fiziksel bellekteki çerçeve numarası = 5

Çerçeve numarası bulunduktan sonra, fiziksel adres, aşağıdaki eşitlik ile bulunur.

Fiziksel adres =(çerçeve numarası(f)*çerçeve boyu) + offset(d)

Physical address= (frame_number * frame_size) + offset

Fiziksel adres=5*4+0=20 = 10100



Örnek-1 :

b)

mantıksal adres : 3 = 0011

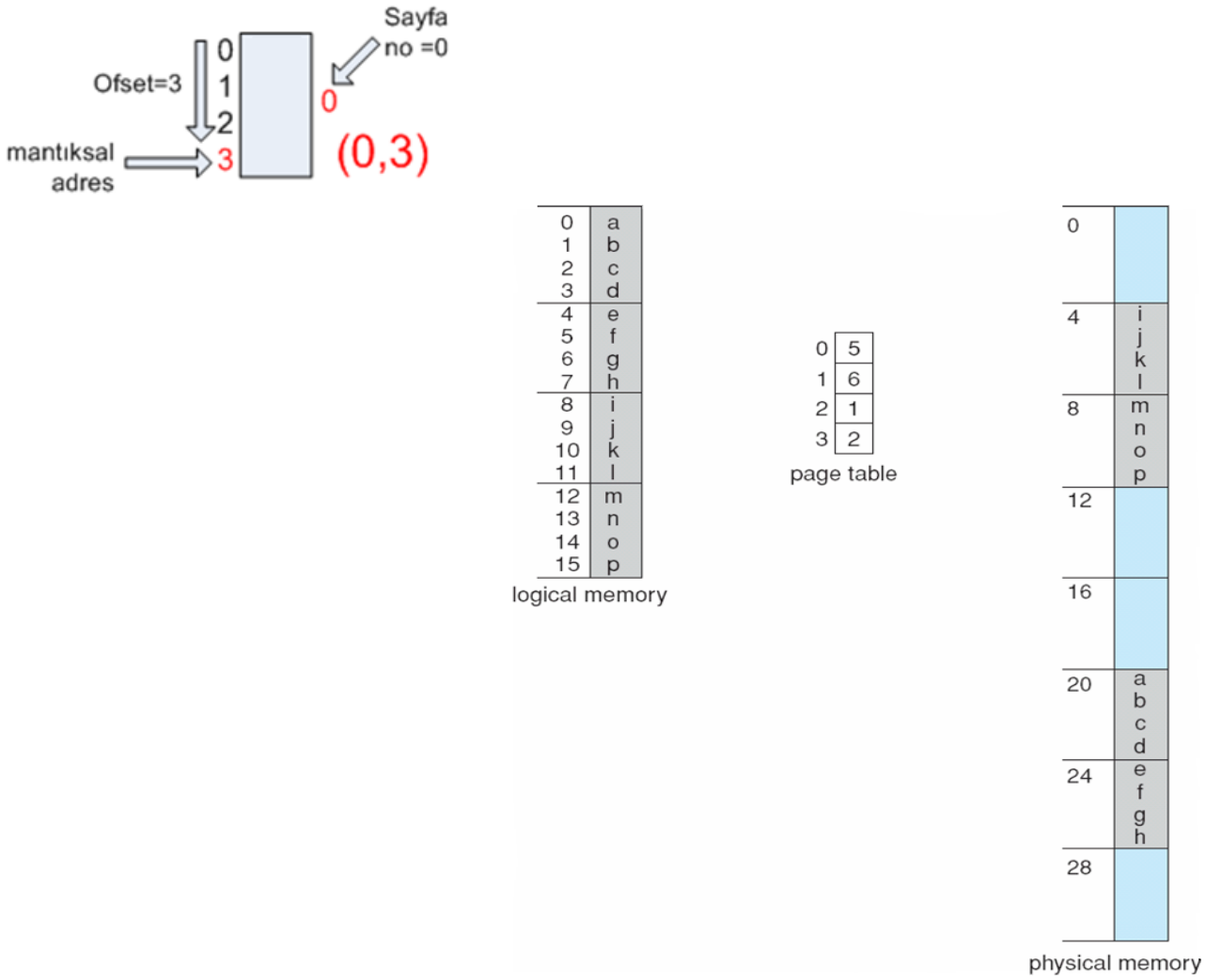
sayfa numarası = 0

sayfa ofseti=3

0 numaralı sayfanın bulunduğu çerçeve numarası = 5

Fiziksel adres =çerçeve numarası (f)*çerçeve boyu + offset (d)

Fiziksel adres=5*4+3=23 = 10111



Örnek-1 :

c)

mantıksal adres : 4 = 0100

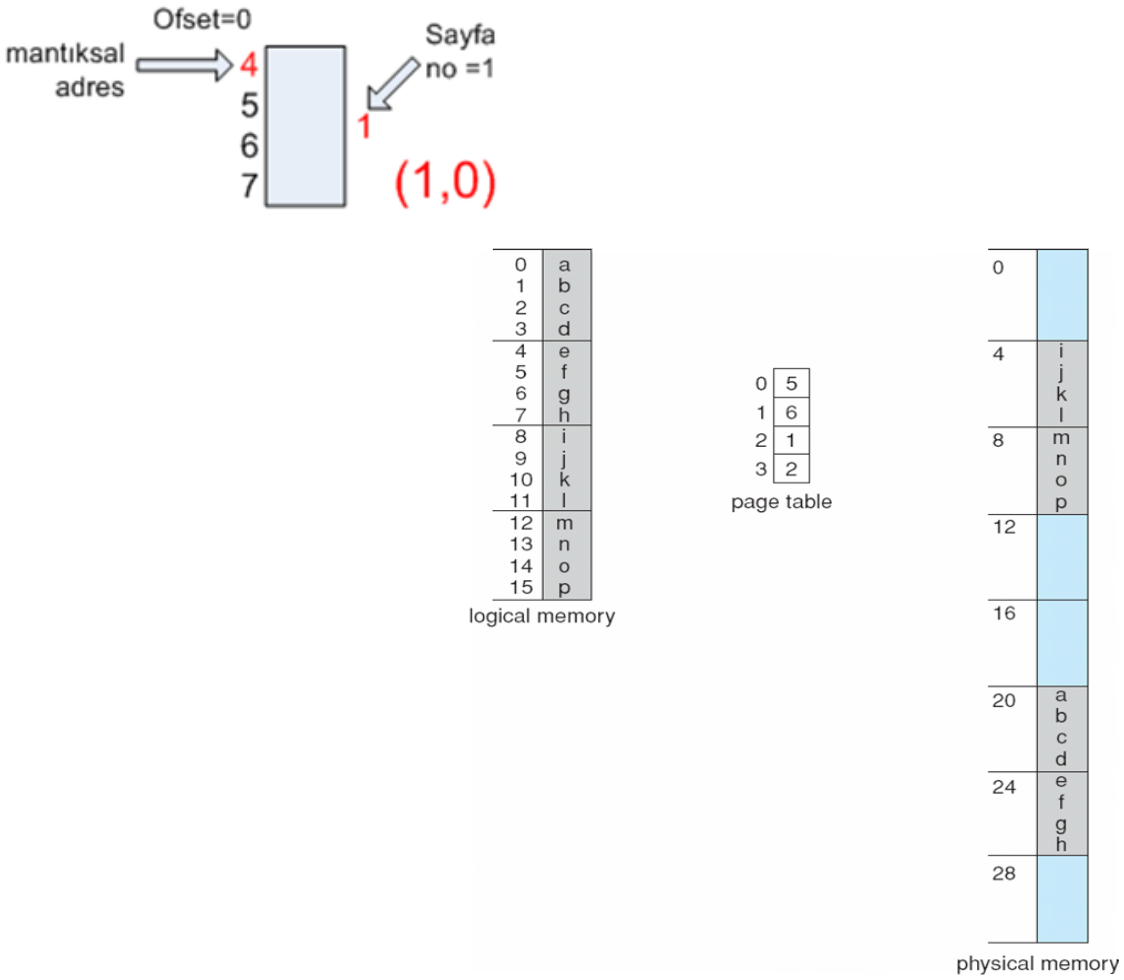
sayfa numarası = 1

sayfa ofseti = 0

1 numaralı sayfanın bulunduğu çerçeve numarası = 6

Fiziksel adres =çerçeve numarası (f)*çerçeve boyu + offset (d)

Fiziksel adres=6*4+0=24 = 11000



Örnek-1 :

d)

mantıksal adres : 13 = 1101

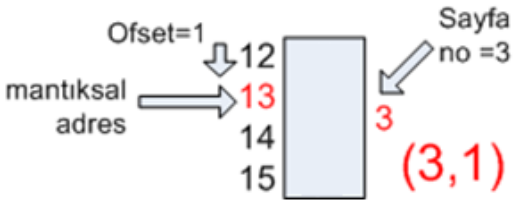
sayfa numarası = 3

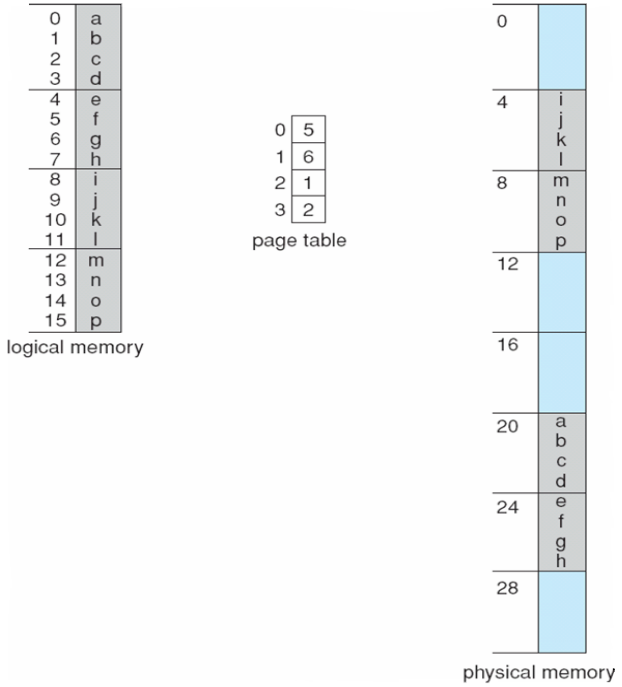
sayfa ofseti = 1

3 numaralı sayfanın bulunduğu çerçeve numarası = 2

Fiziksel adres =çerçeve numarası (f)*çerçeve boyu + offset (d)

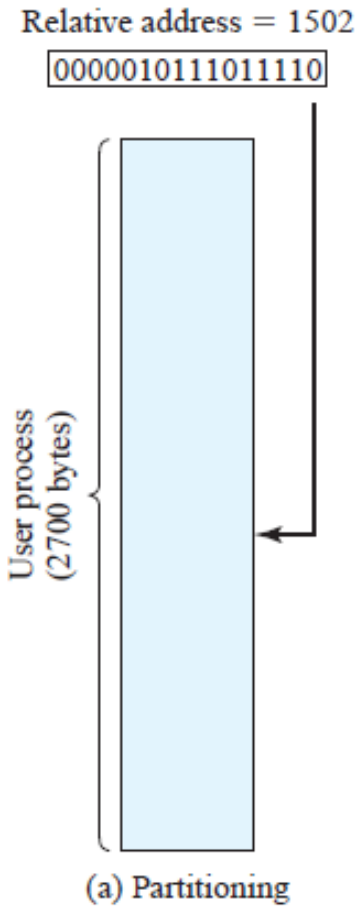
Fiziksel adres=2*4+1=9 = 01001





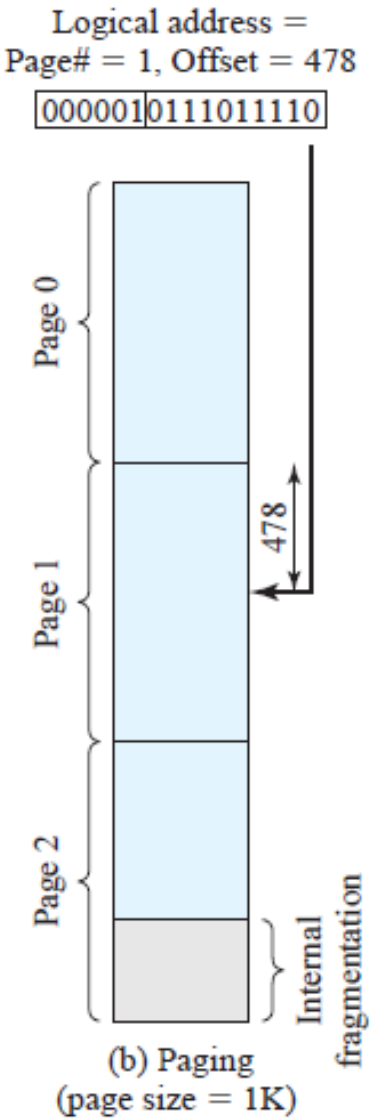
Örnek-2 :

- Mantıksal adres uzayının 16 bit uzunluğunda ve sayfa boyutunun 1KB olduğu varsayılsın.
- Program başına göre bağıl adresi 1502 olan bir mantıksal adresin hangi sayfada olduğunu ve ofset değerini bulunuz.



Örnek-2 :

- 16 bit ile 64KB'lik bir adres uzayı elde edilir. Sayfa boyu 1KB olduğundan mantıksal adres uzayında 64 tane sayfa (page) mevcuttur. 64 sayfa 6 bit ile temsil edilebilir, geriye kalan 10 bit de ofset değeridir.
- $(1502)_{10}$ adresinin binary karşılığı $(\mathbf{0000010111011110})_2$'dir.
- **000001** $\Rightarrow$ sayfa_no=1
- **0111011110** $\Rightarrow$ ofset=478
- En anlamlı 6 bit sayfa numarası olacağından, bu mantıksal adres, birinci sayfada ve ofset değeri 478 olan mantıksal adresdir.



Etkin sayfalar ön belleği
Translation Lookaside Buffer (TLB)

- Mantıksal bir sayfanın adresi fiziksel adrese eşleştirildiğinde sayfa tablosundaki satırlara ulaşımın tekrarı ve dolayısıyla o eşleştirmenin yakın bir zamanda tekrar kullanılma ihtimali yüksektir.
- Etkin sayfalar ön belleği (Translation Lookaside Buffer-TLB) yakın zamanda kullanılan sayfa tablosu satırlarını tutan ve mantıksal-fiziksel adres eşleştirmelerinin izini süren bir ön bellektir.
- TLB Sanal Bellek kullanılıyorsa gereklidir.
- TLB, mantıksal-fiziksel adreslerin eşleşmesini daha hızlı bir bellekte saklayarak erişimin hızlı olmasını sağlar.

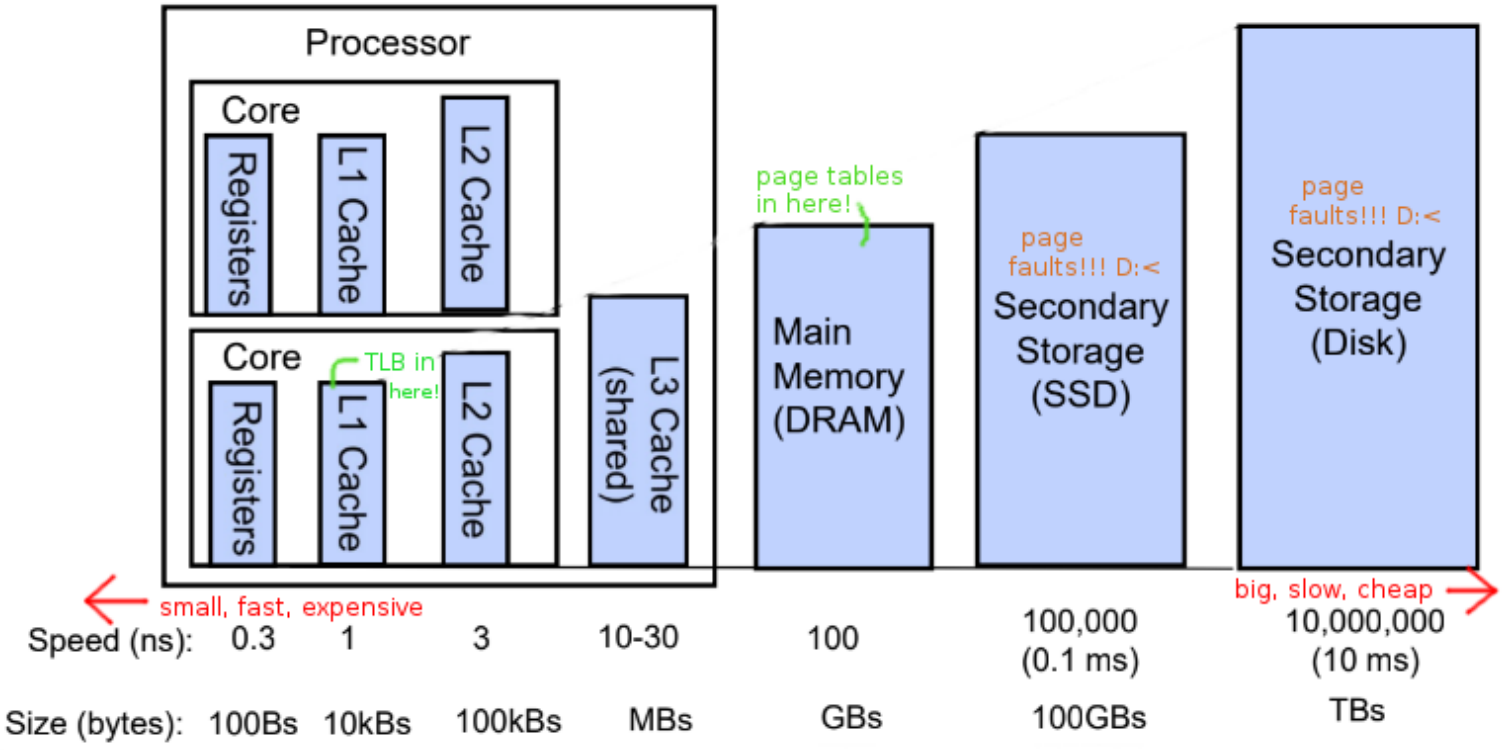
Etkin sayfalar ön belleği
Translation Lookaside Buffer (TLB)

- TLB, sayfalama işleminin (paging) hızını arttırmaya yarayan bir ön bellek bölümüdür.
- Mantıksal adresin fiziksel adrese eşleştirilmesi sırasında Fiziksel Ana bellekte saklanan sayfa tablosunda çerçeve (frame) adresini ararken kaybedilen süreyi önlemek için kullanılan özel, küçük ve hızlı bir ön bellektir.
- TLB'nin tutulduğu ön belleğe erişim Ana belleğe erişimden daha hızlıdır.
- TLB sayfa tablosu eşleştirmelerinin sadece bir kısmını tutan bir ön bellektir (64 ila 1,024 girdi).
- Mantıksal adresin fiziksel adrese eşleştirilmesi gerektiğinde TLB, MMU tarafından kullanılır.

Etkin sayfalar ön belleği
Translation Lookaside Buffer (TLB)

- Bir ana bellek erişimi gerektiğinde, CPU önce TLB'ye bakar. Eğer aranan mantıksal adres TLB'de bulunursa (TLB hit), fiziksel adres hızlı bir şekilde elde edilir.
- Eğer TLB'de bulunamazsa (TLB miss), CPU sayfa tablosuna bakar ve gerekli eşleştirme işlemi yapılarak sonuç TLB'ye eklenir.
- TLB, sayfa tablosu erişimlerini büyük ölçüde hızlandırır çünkü sayfa tablosu erişimi, ana bellek erişim hızlarına kıyasla oldukça yavaş olabilir.
- TLB hit oranı yüksek olduğunda, sistem performansı önemli ölçüde artar. TLB miss durumunda ise ana bellek erişim gecikmeleri yaşanır.

Etkin sayfalar ön belleği
Translation Lookaside Buffer (TLB)

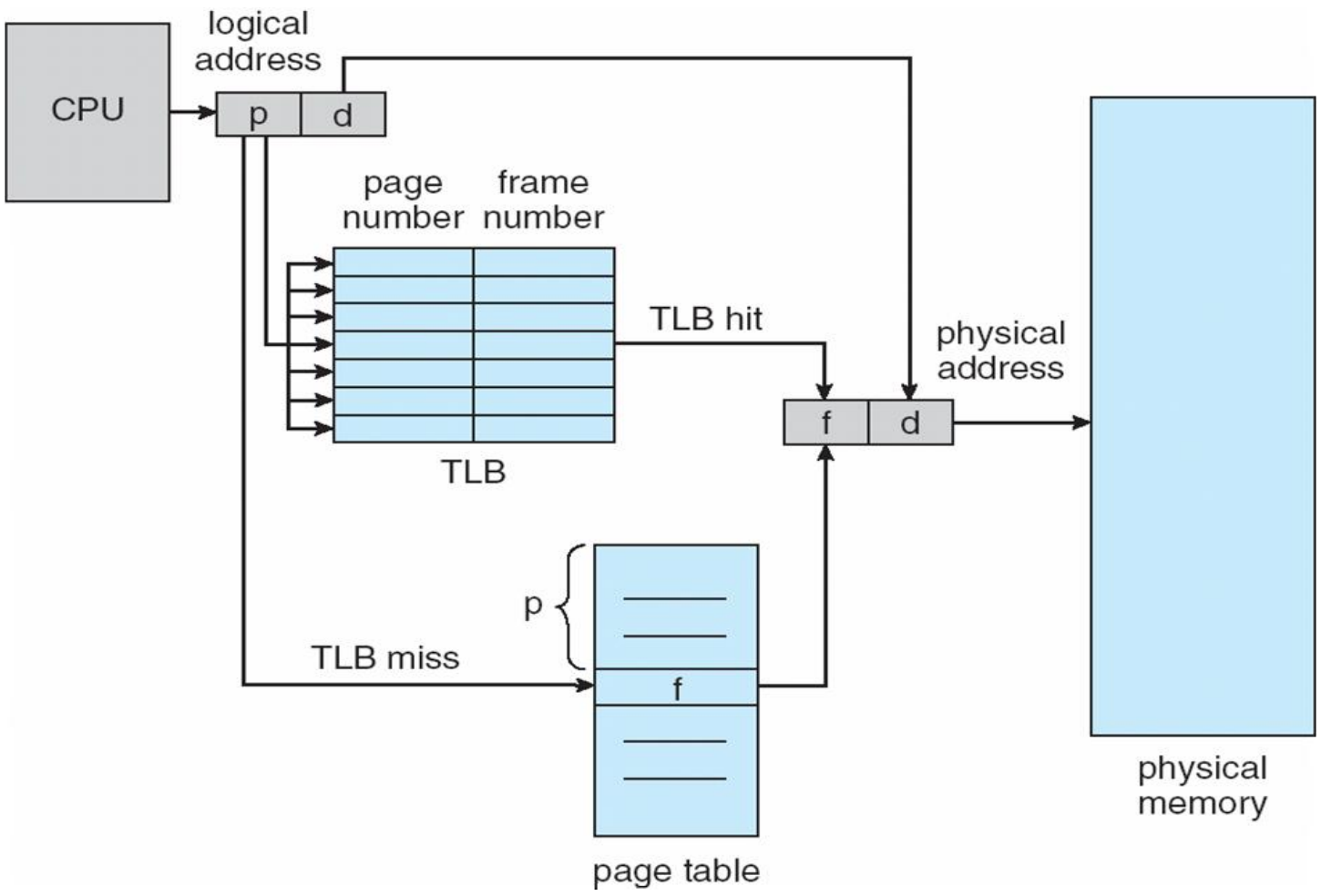


TLB, CPU ve Ana bellek arasında yerleşiktir. Sayfa tablosu ise Ana bellekte yerleşiktir.
TLB mantıksal-fiziksel adreslerin eşleştirilmesini hızlı bir bellekte tutarak mantıksal-fiziksel adres eşleştirmenin hızını artırır.

Etkin sayfalar ön belleği
Translation Lookaside Buffer (TLB)

CPU tarafından mantıksal bir adres tanımlandığında bu adresin sayfa numarası TLB'ye iletilir.
Sayfa numarası burada bulunursa (TLB hit) çerçeve adresi hızlı bir biçimde kullanılarak belleğe ulaşılır.

TLB miss oluşursa (sayfa numarası TLB de yoksa) ana bellekteki sayfa tablosuna gidilerek çerçeve adresi bulunur ve bu TBL'ye de eklenir ki böylece bir daha ihtiyaç olduğunda bu adres hızlı bir biçimde bulunabilir.
Eğer TLB alanı dolarsa, işletim sistemi buradaki en az kullanılan girdi satırlarını bularak bunların yerine yeni girdileri ekleyecektir.



Sayfa Tablolama Yapıları

- Sayfa tablolarının boyutu, çerçeve (frame) boyutundan büyük olduğu durumda sayfa tablosu ana bellekte tek bir çerçevede saklanamaz.

- Büyük sayfa tablolarına karşı aşağıdaki yöntemler önerilmiştir:

- **Hiyerarşik Sayfalama (Hierarchical Paging)**

- **Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)**

Hiyerarşik Sayfalama Yapısı (Hierarchical Paging)

- Modern bilgisayar sistemleri çok büyük mantıksal adres alanlarını kapsamaktadır. (2^{32} ve 2^{64}) bu durumda sayfa tablosu da boyut olarak çok büyük olmaktadır.
- Örneğin, 32 bitlik (2^{32} byte) mantıksal adres alanında,
- sayfa büyüklüğü 4 KB (2^{12}) kullanılırsa yaklaşık 1 milyon ($2^{32}/2^{12} = 2^{20}$),
- sayfa büyüklüğü 1 KB (2^{10}) kullanılırsa yaklaşık 4 milyon ($2^{32}/2^{10} = 2^{22}$)
- sayfa tablosu girdi satırına ihtiyaç olacaktır.

- **Sayfa Tablosu Boyutu = Sayfa sayısı* Sayfa tablosunun herbir girdi satırının boyutu**

Hiyerarşik Sayfalama Yapısı (Hierarchical Paging)

- 64 bitlik (2^{64} byte) mantıksal adres alanında ise, 4 KB sayfalar ile 2^{64} / 2^{12}= 2^{52} girdili sayfa tablolarına ihtiyaç olacaktır.

- Bu değer de çok büyük olduğu için, sayfa tablosunu bütün olarak ana bellekte tutmak mümkün olmamaktadır, bu yüzden sayfa tablosunu parçalara ayırmak daha uygun olmaktadır.
- Hiyerarşik Sayfalama, mantıksal adres alanını çoklu sayfa tablolarına böler.

- İki Seviyeli, üç seviyeli, vb. Hiyerarşik Sayfalama uygulanabilir.

Hiyerarşik Sayfalama Yapısı (Hierarchical Paging)

- Hiyerarşik sayfalamada,
- Çerçeve (frame) boyutundan daha büyük boyuta sahip sayfa tablosu birkaç bölüme ayrılmıştır.
- Her parçanın boyutu, muhtemelen son parça dışında çerçeve boyutuyla aynıdır.
- Sayfa tablosunun bölümleri daha sonra ana belleğin farklı çerçevelerinde saklanır.
- Bu yöntemde, sayfa tablosunun bölümlerini saklayan çerçevelerin kaydını tutmak için başka bir sayfa tablosu tutulur.
- Sonuç olarak, sayfa tablolarının hiyerarşisi oluşturulur.
- Tüm sayfa tablosu bölümlerinin her birinin tek bir çerçevede saklanabileceği düzeye ulaşılana kadar çok düzeyli sayfalama yapılır.

Hiyerarşik Sayfalama Yapısı (Hierarchical Paging)

- Sayfa tablosunun boyutu bir fiziksel bellek çerçevesinin boyutundan küçük olduğunda problem olmaz çünkü sayfa tablosu doğrudan ana belleğin bir çerçevesine yerleştirilir. Böylece sayfa tablosuna doğrudan erişilir.

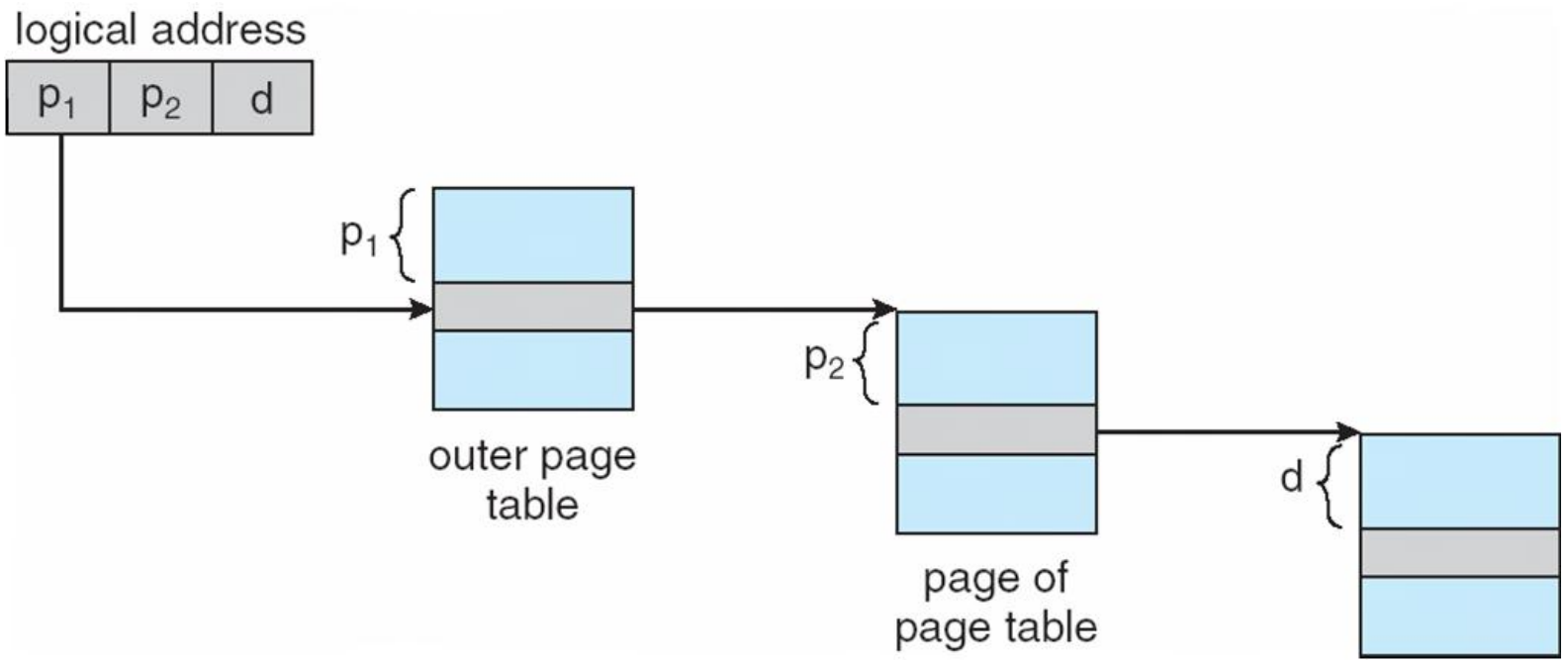
- Ancak sayfa tablosunun boyutu fiziksel bellek çerçevesinin boyutundan büyükse, sayfa tablosu birkaç seviye (level) olarak ayrılacak ve bu ayrı seviyeler ana bellekte farklı çerçevelerde saklanacaktır.

32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

- 32 bitlik bir mantıksal adres uzayı (2^{32}), 1KB'lık (2^{10}) sayfalara bölündüğünde mantıksal adresteki sayfa numaraları 22 bitlik , sayfa ofseti ise 10 bitlik olmaktadır.
- Hiyerarşik Sayfa Tablosu kullanımında, Sayfa tablosuna erişmek için de sayfa tablosu kullanılması gerekeceği için 22 bitlik sayfa numarası da 12 bitlik sayfa numarası ve 10 bitlik sayfa ofseti olarak tekrar ikiye ayrılacaktır.
- Mantıksal adres aşağıdaki gibi olur.
- Burada p1 "outer page table" indeksini (ofset), p2 ise "page of page table (inner page table)" indeksini (ofset) ifade etmektedir.

page number		page offset
p_1	p_2	d
12	10	10

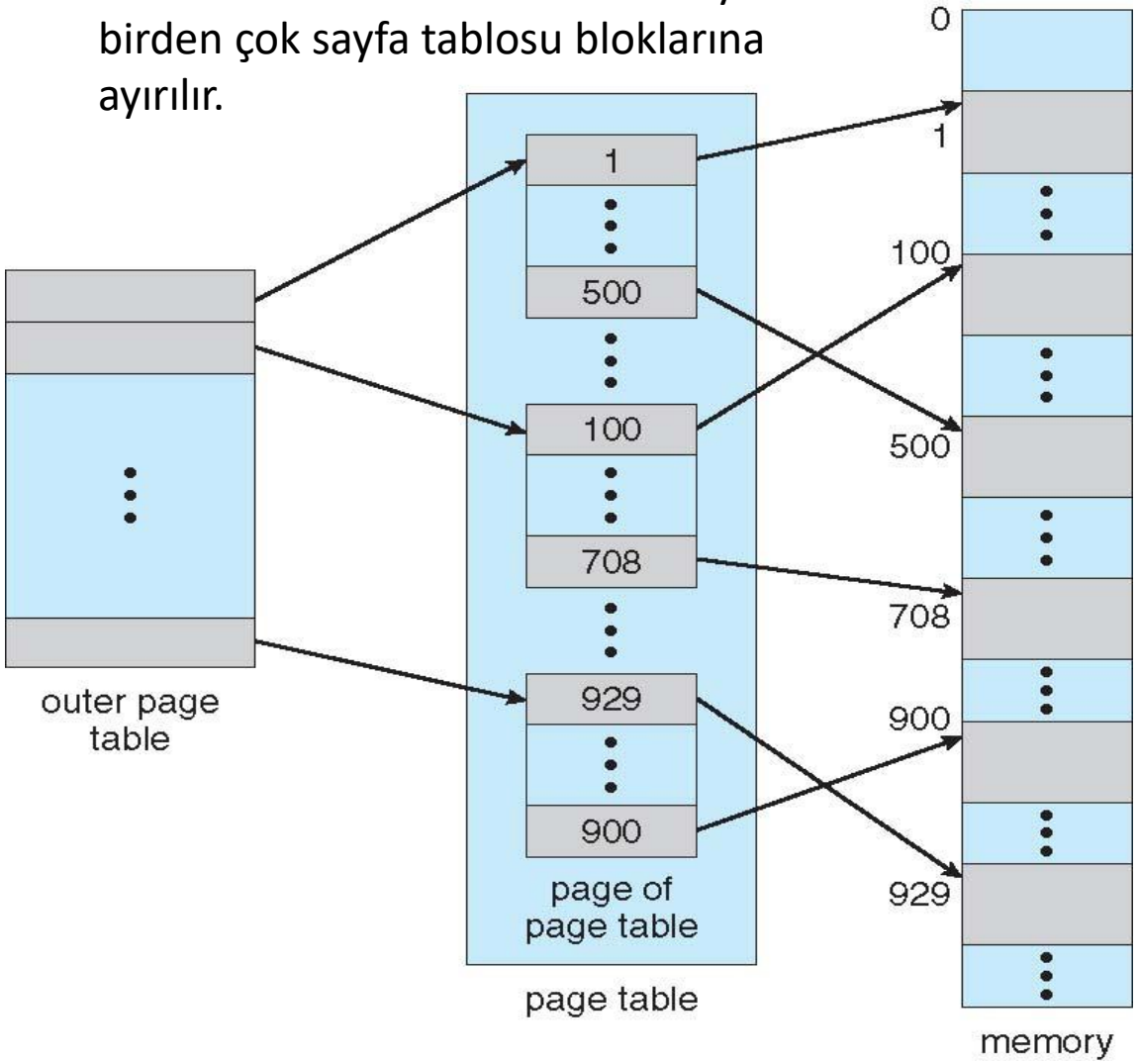
İki Seviyeli Sayfalama (2-Level Page Tables)



İki Seviyeli Sayfalama (2-Level Page Tables) için bir "Outer Page Table" devreye giriyor.
Mantıksal adresteki p1: "Outer page table" için bir ofset değeri içerir (Outer Level Index).
Mantıksal adresteki p2: "Page of page table (inner page table)" için bir ofset değeri içerir (Inner Level Index).
"page of page table (inner page table)" içinde bir sayfayla aynı boyutta sabit boyutlu bloklar bulunur. "Outer page table" bu sayfa tablosu bloklarının base adreslerini içerir.

İki Seviyeli Sayfalama (2-Level Page Tables)

Mantıksal adres alanı farklı düzeylerde birden çok sayfa tablosu bloklarına ayırılır.



İki Seviyeli Sayfalama (2-Level Page Tables)

Outer page table: "Inner page table" içinde bir sayfayla aynı boyutta sabit boyutlu sayfa tablosu blokları bulunur. "Outer page table", "Inner page table" içindeki sayfa tablosu bloklarının başlangıç adreslerini (base address) içerir. "Outer page table" içindeki her giriş satırı, "page of page table" içinde bir sayfa tablosu bloğuna işaret eder.

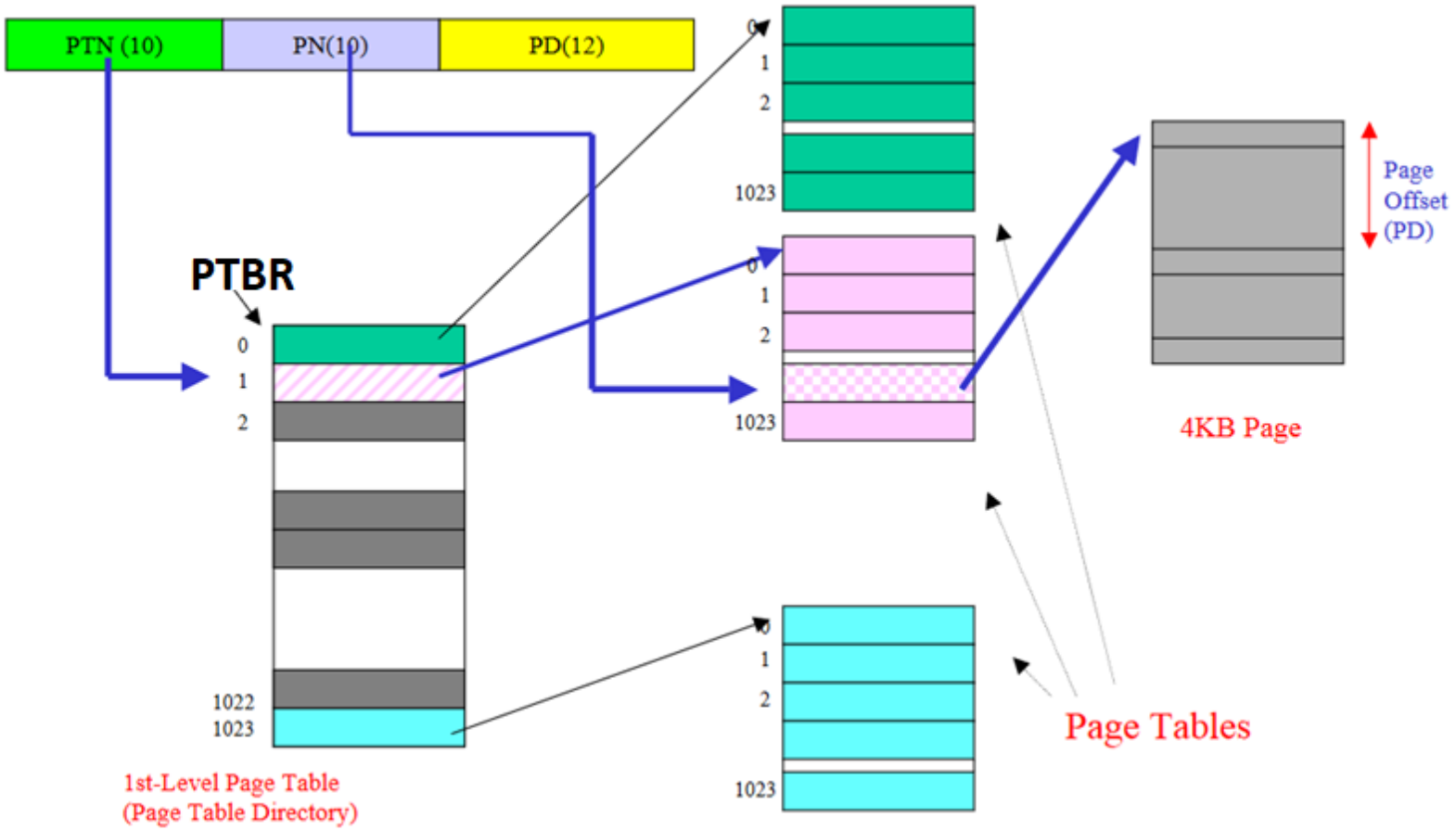
Page of page table (inner page table): "Page of page table" içindeki sayfa tablosu blokları hangi sayfanın fiziksel adresteki hangi çerçevede tutulduğunun bilgisini içerir. Herbir blok içindeki sayfa tabloları fiziksel bellekte farklı bir çerçeve (frame) içinde saklanır.

32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

Sayfa tablosu bellekte saklanır. Fakat,

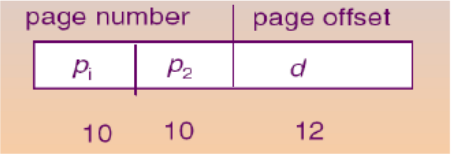
Page Table Base Register (PTBR), Outer page table'nin taban adresini işaret eder.

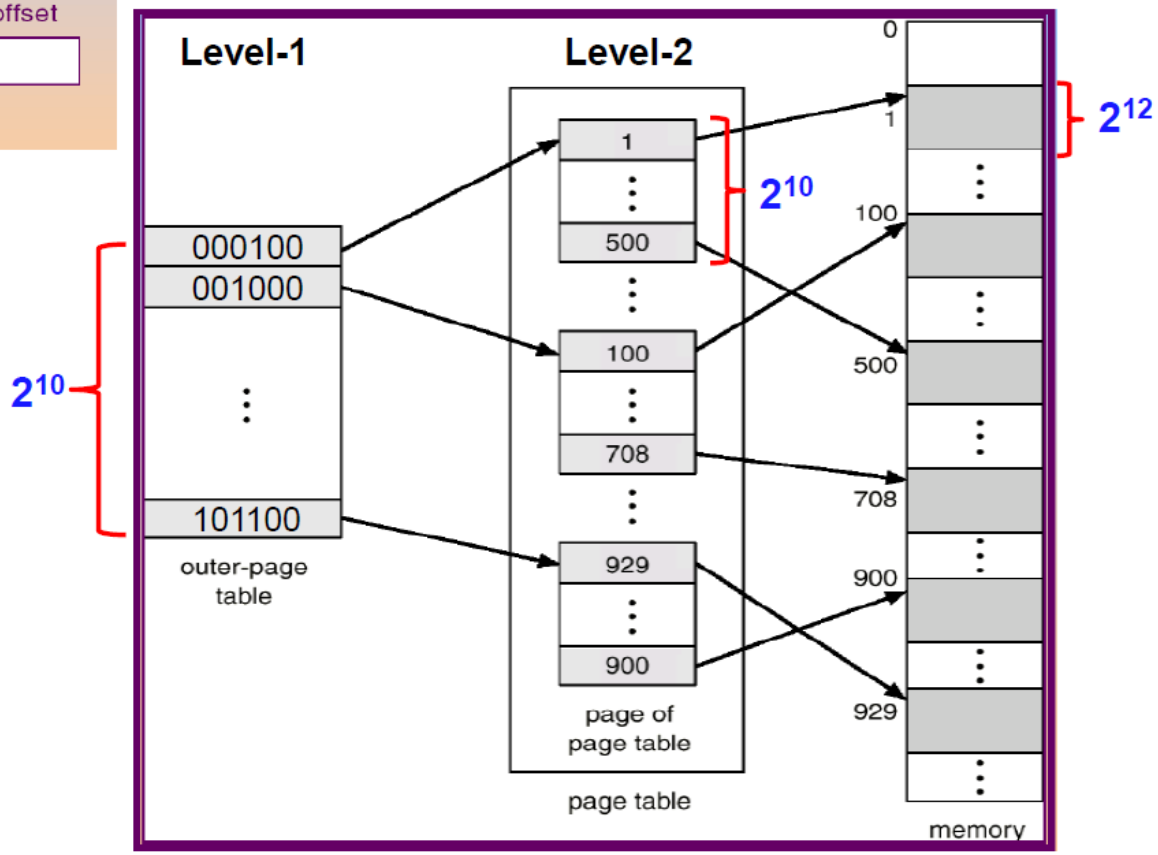
İşletim sistemi, bu PTBR değerini process kontrol bloğunda (PCB) saklar.



32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

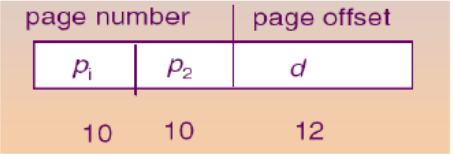
- 32 bitlik bir mantıksal adres uzayı (2^{32}), **4KB'lık** (2^{12}) sayfalara bölündüğünde mantıksal adresteki sayfa numaraları 20 bitlik , sayfa ofseti ise 12 bitlik olmaktadır.
- Hiyerarşik Sayfa Tablosu kullanımında, Sayfa tablosuna erişmek için de sayfa tablosu kullanılması gerekeceği için 20 bitlik sayfa numarası da 10 bitlik sayfa numarası p1 ve 10 bitlik sayfa numarası p2 olarak tekrar ikiye ayrılacaktır.

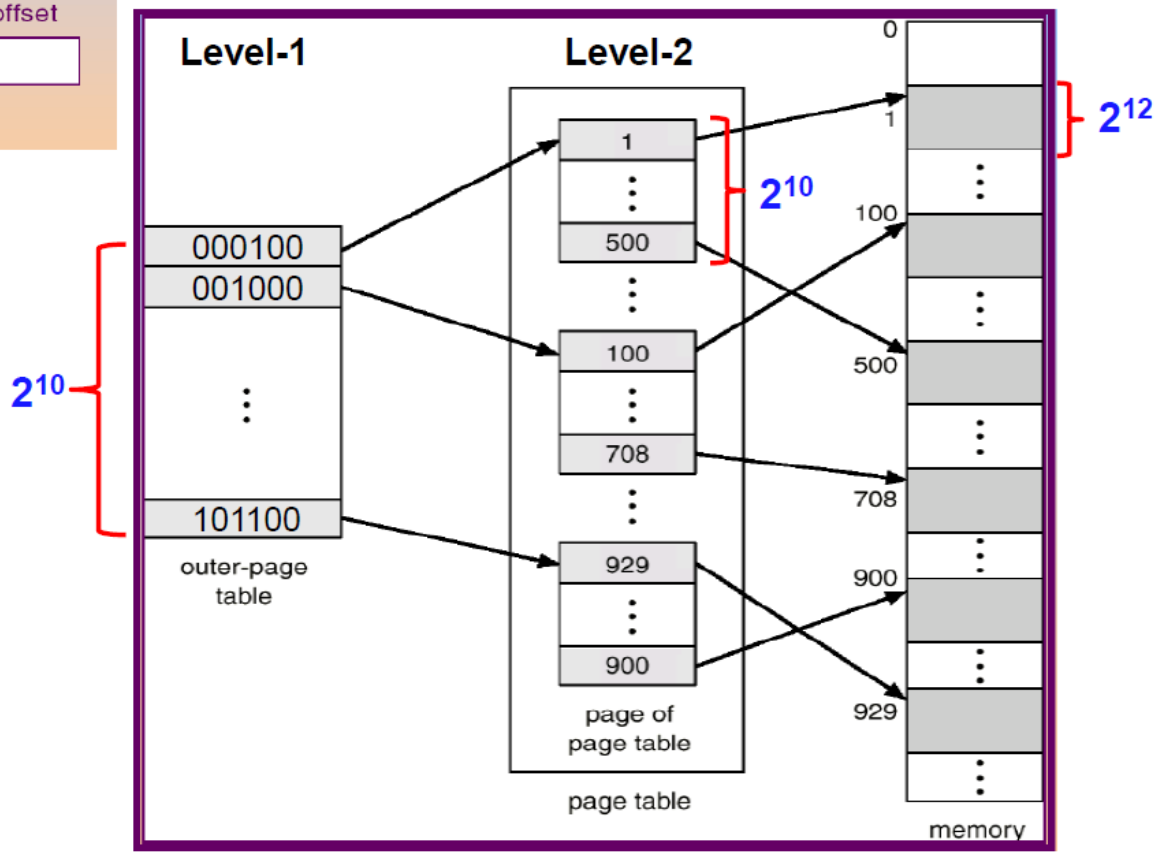




32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

- 32 bitlik bir mantıksal adres uzayı (2^{32}), ve 4KB'lık (2^{12}) sayfa uzunluğu için;
- Sayfa tablosunu bir sayfayla aynı boyutta sabit boyutlu sayfa tablosu bloklarına ayırmak gerekiyor.
- Örnekte: Sayfa tablosunun herbir satır girdisinin 4 bayt olduğu kabul edilirse.
- Sayfa Tablosunun toplam boyutu = Sayfa sayısı* girdi satırı boyutu = 2^{20}*4 bayt = 4MB
- Böylece, "Page of page table (inner page table)" içinde 4MB/4KB = **2^{10} = 1024** tane sayfa tablosu blokları olacaktır.
- "Outer page table" bu sayfa tablosu bloklarının fiziksel bellekteki base adreslerini içerir. Dolaysıyla bu tabloda 2^{10} = 1024 girdi satırı bulunmalıdır.





32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

- 32-bitlik bir sanal adresimiz ve 2KB'lık sayfalar kullandığımızı varsayalım.
- İki seviyeli sayfa tablosunun için;
- Mantıksal Adres: 32-bit
- Sayfa Boyutu: 2KB (2^{11} byte)
- Bu hiyerarşik iki seviyeli sayfalama için:
- Outer page table: İlk 11 bit (2^{11} = 2048 giriş satırı)
- Inner page table: Sonraki 10 bit (2^{10} = 1024 blok)
- Sayfa İçi Ofset (Page Offset): Son 11 bit (2^{11} = 2048 byte)

- [Outer page table][Inner page table][Page offset]
- [11 bit] [10 bit] [11 bit]

32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalama

Outer page table

| 0 | -> Inner Page Table_0 Başlangıç Adresi
| 1 | -> Inner Page Table_1 Başlangıç Adresi

| ... |
| ... |

|2047| -> Inner Page Table_2047 Başlangıç Adresi

32-bit Mantıksal Adres Uzayı ve İki Seviyeli Sayfalam

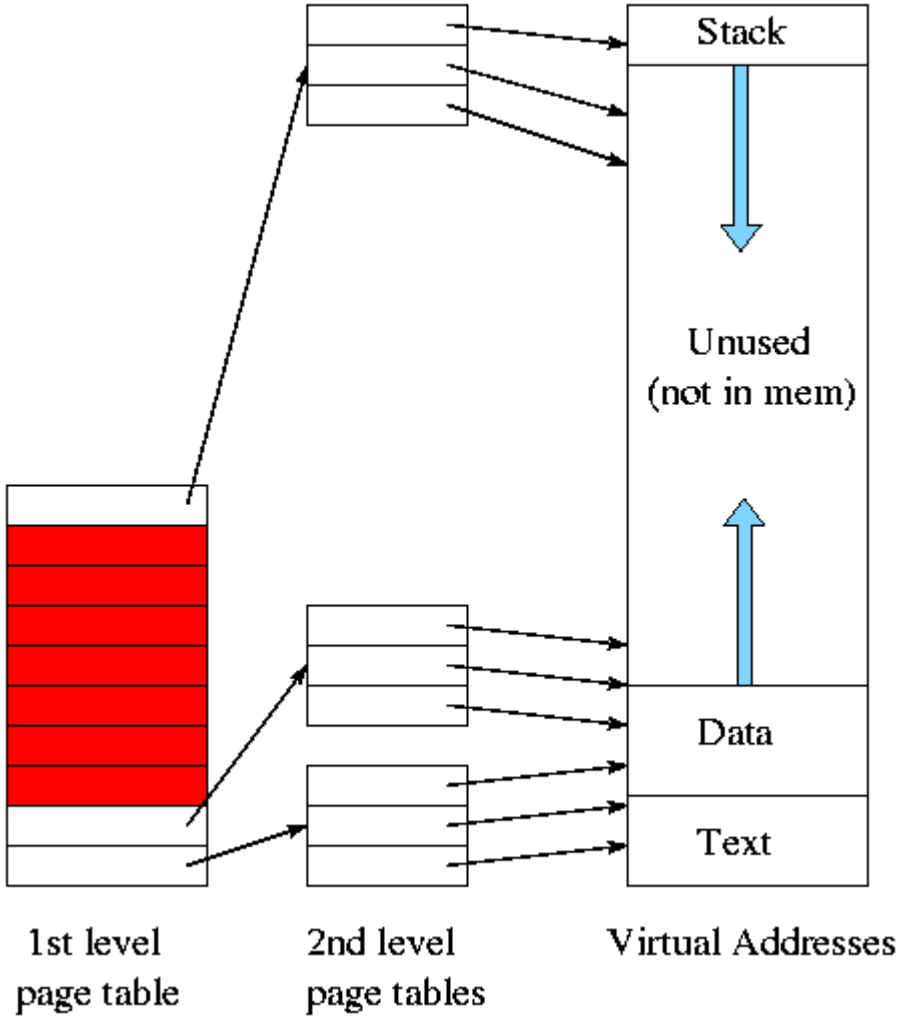
Page of page table (inner page table)

| Sayfa Tablosu_0 |
| 0 | -> Sayfa_0'ın tutulduğu Çerçevenin Başlangıç Adresi
| 1 | -> Sayfa_1'ın tutulduğu Çerçevenin Başlangıç Adresi
| ... |
| 1023 | -> Sayfa_1023'ın tutulduğu Çerçevenin Başlangıç Adresi

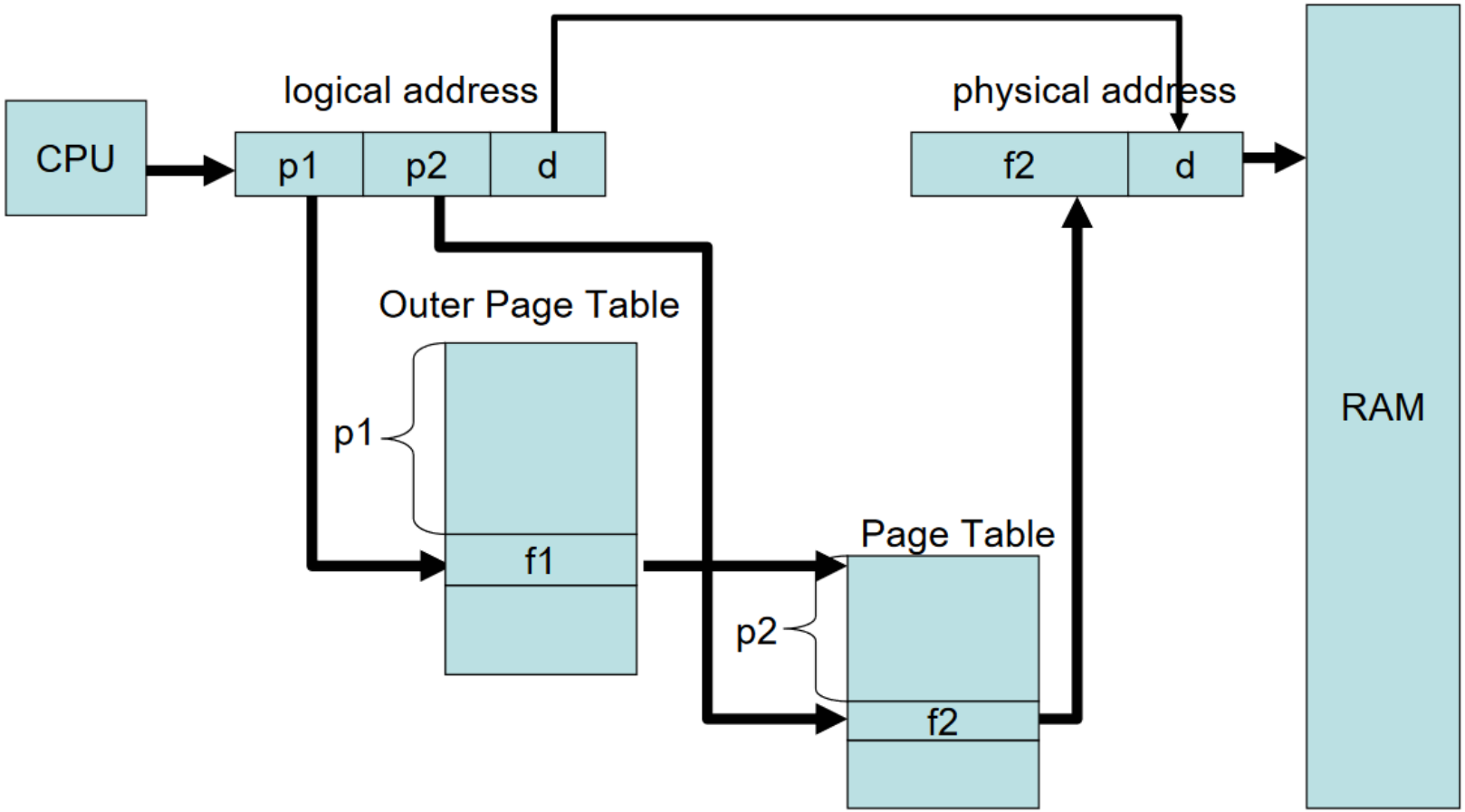
| Sayfa Tablosu_1 |
| 0 | -> Sayfa_0'ın tutulduğu Çerçevenin Başlangıç Adresi
| 1 | -> Sayfa_1'ın tutulduğu Çerçevenin Başlangıç Adresi
| ... |
| 1023 | -> Sayfa_1023'ın tutulduğu Çerçevenin Başlangıç Adresi

| ... |

| Sayfa Tablosu_2047 |
| 0 | -> Sayfa_0'ın tutulduğu Çerçevenin Başlangıç Adresi
| 1 | -> Sayfa_1'ın tutulduğu Çerçevenin Başlangıç Adresi
| ... |
| 1023 | -> Sayfa_1023'ın tutulduğu Çerçevenin Başlangıç Adresi



İki Seviyeli Sayfalama (2-Level Page Tables) Mantıksal-fiziksel adres dönüşümü



64-bit Mantıksal Adres Uzayı

- 64 bitlik bir mantıksal adres uzayı 4KB'lık (2^{12}) sayfalara bölündüğünde sayfa tablosu
- $2^{64} / 2^{12} = 2^{52}$ girişe sahip olacaktır.
- Eğer 2 seviyeli sayfalama kullanılırsa,

outer page	inner page	page offset
p_1	p_2	d
42	10	12

- Outer page tablosunda 2^{42} giriş var. Bu değer de çok büyük.
- Bir çözüm, ikinci bir outer page tablosu daha eklemektir.

Üç seviyeli Sayfalama (3-Level Page Tables)

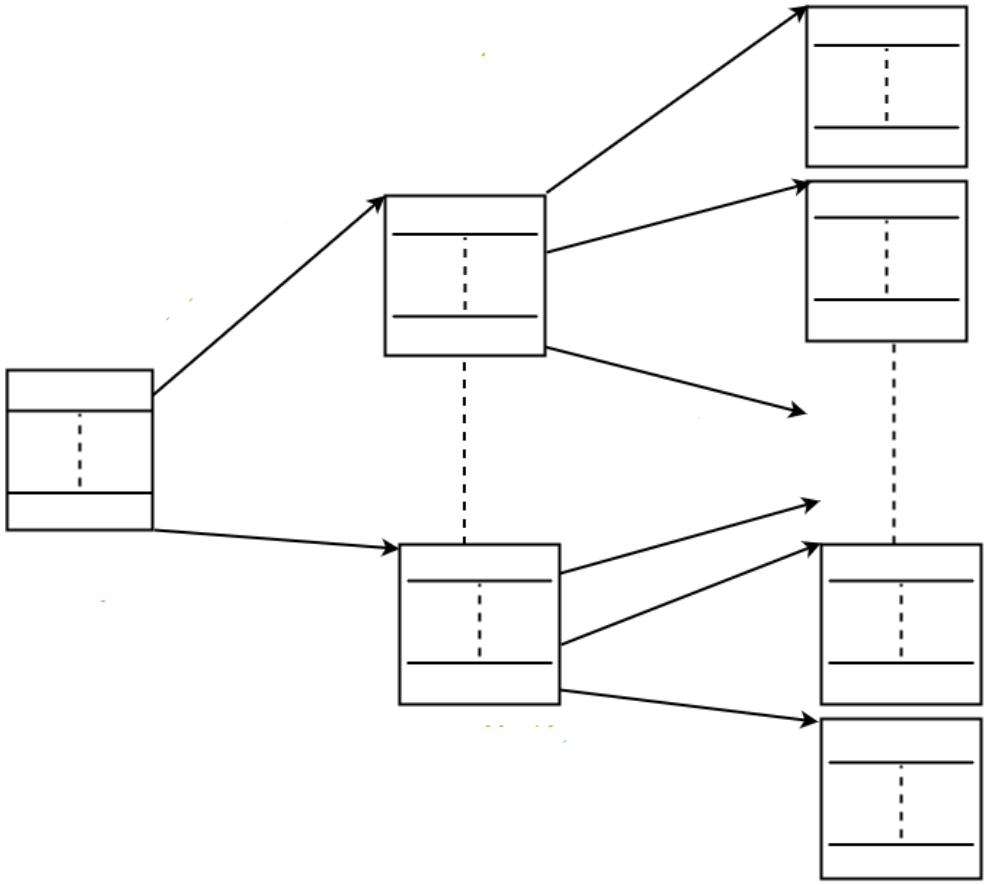
İki seviyeli Sayfalama

outer page	inner page	offset
p_1	p_2	d
42	10	12

Üç seviyeli Sayfalama

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Üç seviyeli Sayfalama (3-Level Page Tables)



Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- İşletim Sistemlerinin çoğu, her process için ayrı bir sayfa tablosu uygular,
- yani işletim sisteminde çalışan "n" process için, bellekte saklanan "n" sayıda sayfa tablosu vardır.

- Bazen bir process boyut olarak çok büyük olduğunda, process'in büyüklüğü ile sayfa tablosunun boyutu da önemli ölçüde artar.

- Ters Çevrilmiş Sayfa Tablosu kullanmanın amacı, geleneksel sayfa tablolarına göre daha az bellek kullanarak mantıksal adresleri fiziksel adreslere çevirmektir.

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- 2 GB boyutunda bir process:

- Sayfa uzunluğu = 512 bayt
- Sayfa tablosu girdi satırı boyutu = 4 bayt $=2^2$ bayt
- Process'deki sayfaların sayısı = 2 GB / 512 bayt = 2^{22}
- Sayfa tablosunun boyutu = $2^{22} * 2^2 = 2^{24}$ bayt

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- 64 bit ve 8kb sayfa boyutu için, 2^{64} / 8kb = 2^{51} tek sayfa tablosu için girdi satırı (entry) sayısı,
- bu nedenle sayfa tablosunun boyutu, 4 bayt girdi satırı (entry) için 2^{53} bayt olacaktır.
- Process başına saklamak için çok büyük bir sayfa tablosu.

- Dolaysıyla, bir işletim sisteminde aynı anda çalışan birden fazla process için, belleğin hatırı sayılır bir kısmının yalnızca sayfa tabloları tarafından doldurulduğu sonucuna varılabilir.

- Çözüm için Ters çevrilmiş sayfa tablosu (Inverted Page Table) kullanılır.

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table) yaklaşımında, **fiziksel bellekteki herbir çerçeve (frame) başına sayfa tablosunda bir girdi bulunmaktadır.**
- Yani Ters Çevrilmiş Sayfa Tablosu , fiziksel bellek çerçevelerinin sayısı kadar girdi içerir.
- Örneğin;
- 2 GB boyutunda fiziksel bellek uzayı $=2^{31}$ bayt
- Sayfa uzunluğu = 8 KB = 2^{13} bayt
- Ters Çevrilmiş Sayfa Tablosu girdi satırı (entry) sayısı = 2^{31} / 2^{13} $=2^{18}$

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

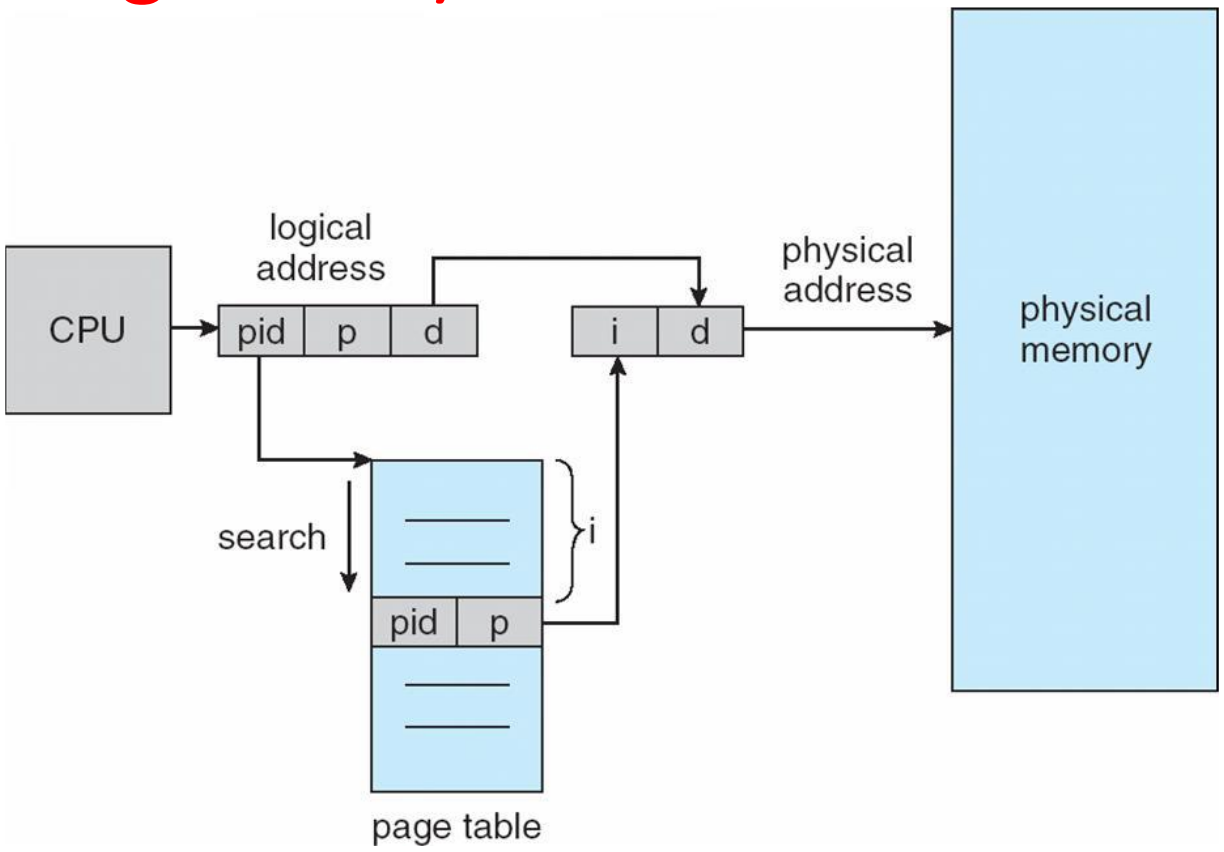
- Ters Çevrilmiş Sayfa Tablosu tüm process'ler için işletim sistemi tarafından sağlanan bir genel sayfa tablosudur.
- Tüm sistemde yalnızca bir adet ters çevrilmiş sayfa tablosu vardır.
- Yani her process için ayrı bir sayfa tablosunun saklanması ortadan kaldırılır ve tüm process'lerin sayfalama bilgilerinin birlikte saklanması için yalnızca sabit bir bellek kısmı gerekir.
- Tüm process'lerin mantıksal adresleri tek bir tabloda yönetildiği için bellek yönetimi daha basit hale gelir.
- Fakat, bir mantıksal adresin fiziksel adrese çevrilmesi için tablo üzerinde arama yapılması gerektiğinden, bu işlem geleneksel sayfa tablolarına göre daha yavaş olabilir.

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- Şimdiye kadar tanımladığımız sayfa tablolarında indeksleme sayfa numarasına göre yapılmaktaydı.

- Halbuki, Ters Çevrilmiş Sayfa Tablosu tekniğinde indeksleme, sayfa numarası yerine çerçeve (frame) numarasına göre yapılır.

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)



pid: process_id, p:page number, d: offset

Her mantıksal adres üç kısımdan oluşur:
< process numarası, sayfa numarası, ofset >

Ters çevrilmiş sayfa tablosundaki her girdi satırı (entry), process numarası (process_id) ve sayfa numarası (page number) bilgilerini içerir.

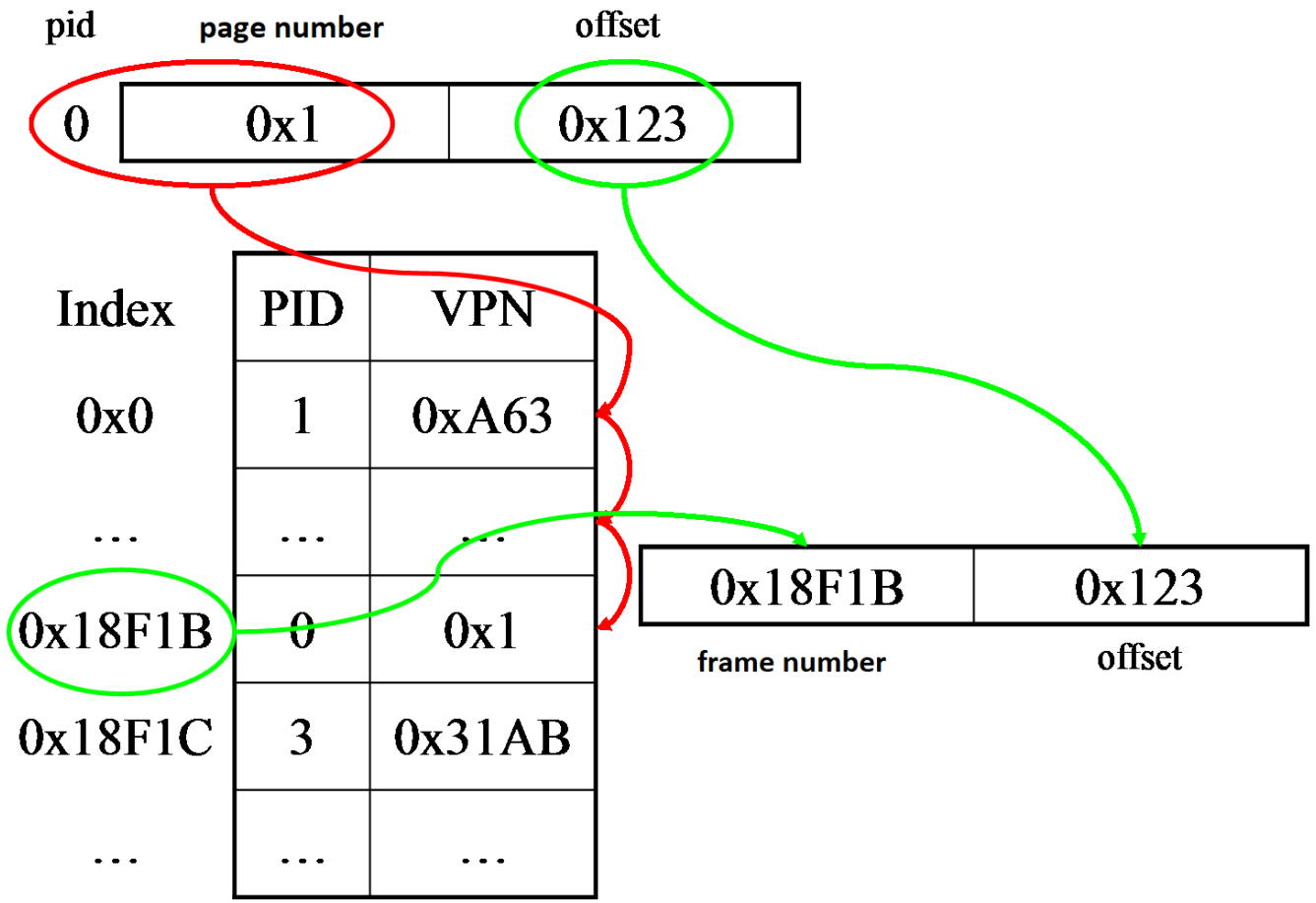
Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- Bir bellek başvurusu olduğunda ilk olarak;
- <process numarası, sayfa numarası> için ters çevrilmiş sayfa tablosunda bu adrese bir karşılık aranır.

- Eğer bu adresin karşılığı sayfa tablosunun i. girdi satırında bulunursa,
- <i, ofset> şeklinde fiziksel adresi oluşturulur.

- Eğer <process numarası, sayfa numarası> için sayfa tablosunda bir karşılık bulunmazsa geçersiz bir adres tanımlaması yapılmış olacaktır.

Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)



Ters Çevrilmiş Sayfa Tablosu (Inverted Page Table)

- Ters Çevrilmiş Sayfa Tablosu kullanımında bellek kullanım israfı önlense de;

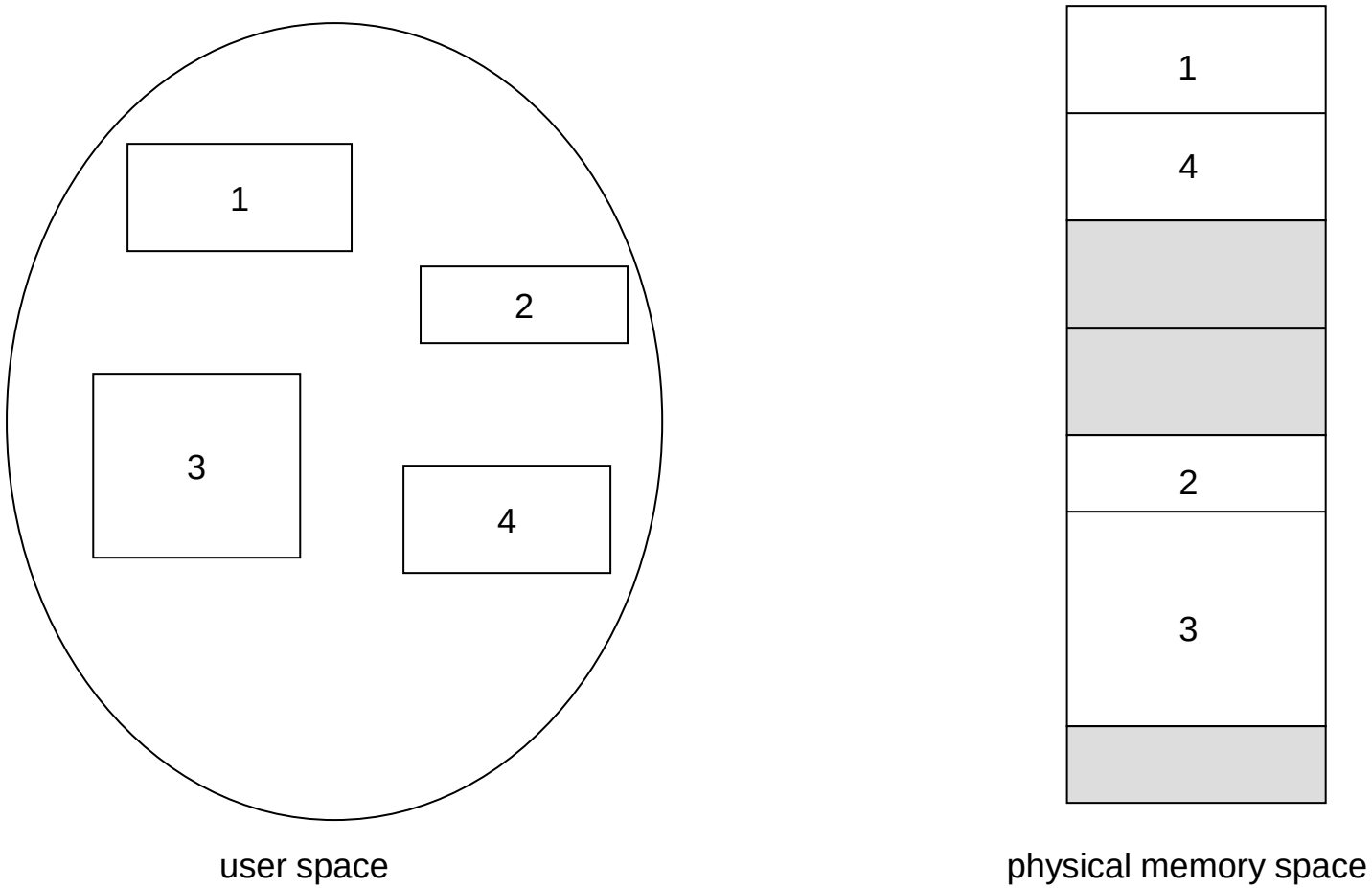
- arama süresi artar,
- mantıksal – fiziksel eşleştirme işlemi çok daha zorlaşır.

Kesimli Bellek Yönetimi (Segmentation)

Kesimli Bellek Yönetimi (Segmentation)

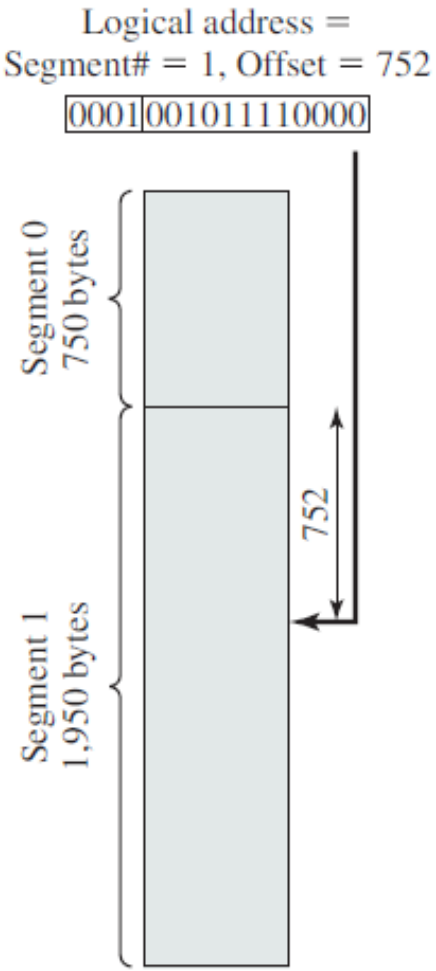
- Kesimli bellek yönetiminde (segmentation) mantıksal adres uzayı sayfalar yerine segmentlerden (kesimlerden) oluşmaktadır.
- Tüm segmentler aynı boyda olmak zorunda değildir.
- Ancak segment boyunun bir üst sınırı bulunur.
- Maksimum segment boyunun (segment size) bilinmesi gerekir.
- Bir programın derlenmesi sırasında derleyici segment atamalarını gerçekleştirir.

Kesimli Bellek Yönetimi (Segmentation)



Kesimli Bellek Yönetimi (Segmentation)

- Bir mantıksal adres, hem segment numarası (s) hem de segment içi ofset değerinden (d) meydana gelmektedir.



Mantıksal Adres (Logical address)

< Segment numarası (s), Segment ofset (d) >

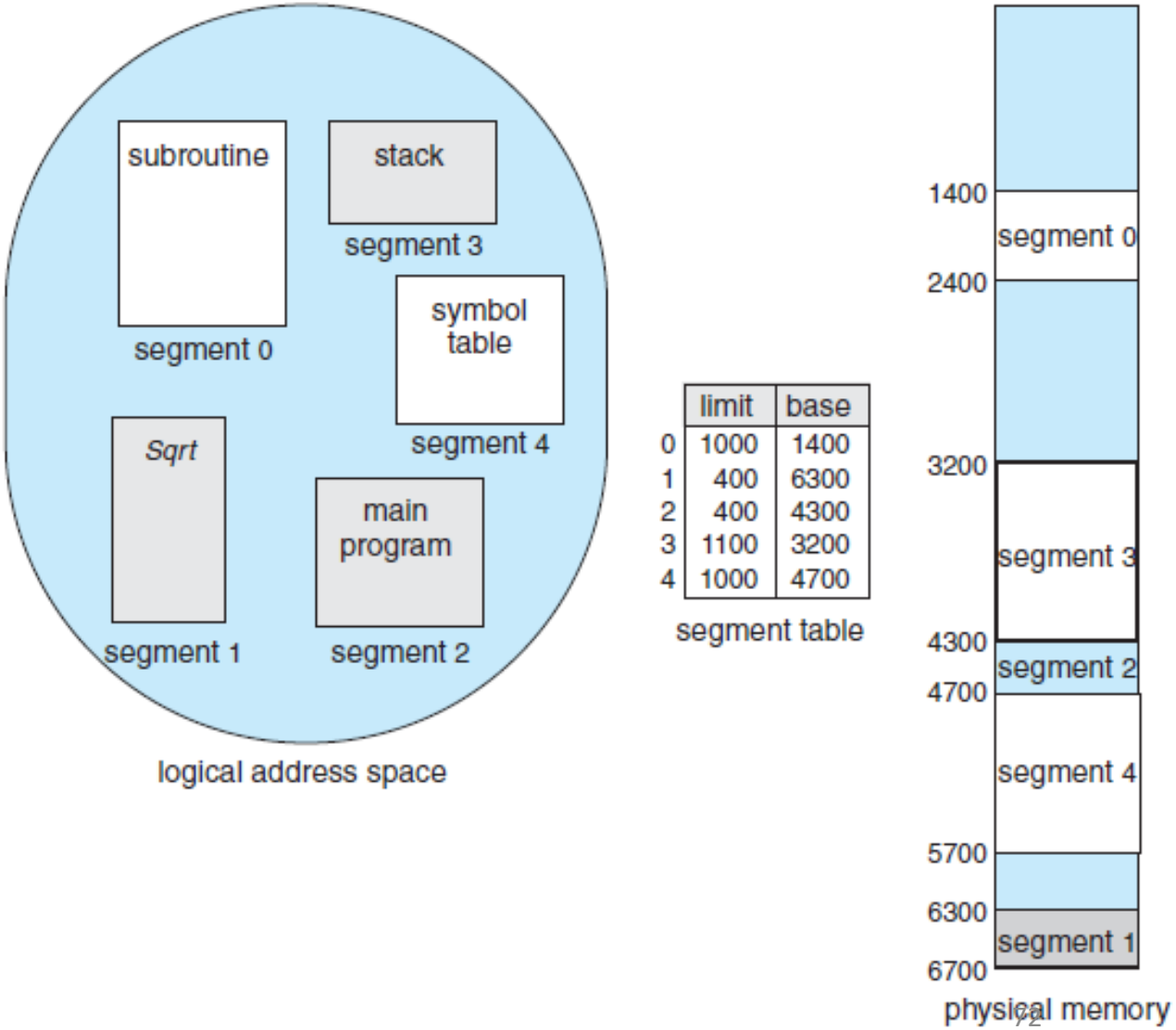
Kesimli Bellek Yönetimi (Segmentation)

- Mantıksal adreslerin fiziksel adreslere eşleştirilmesi için segment tablosuna ihtiyaç vardır.
- Segment tablosu, her segmentin taban (base) adresi ve uzunluğu (limit) bilgisini tutmaktadır.
- base : Bellekte segmentin başlangıç adresini tutar.
- limit : Segmentin uzunluğunu tutar.

- Böylece her segmentin başlama (taban) ve bitiş (tavan) adresi hesaplanabilir.

Kesimli Bellek Yönetimi (Segmentation)

- Mantıksal adres ve Fiziksel bellek uzaylarının yanda göründüğü gibi olduğu varsayılırsa.
- Segment tablosu bu bilgiler kullanılarak elde edilebilir.



Kesimli Bellek Yönetimi (Segmentation)

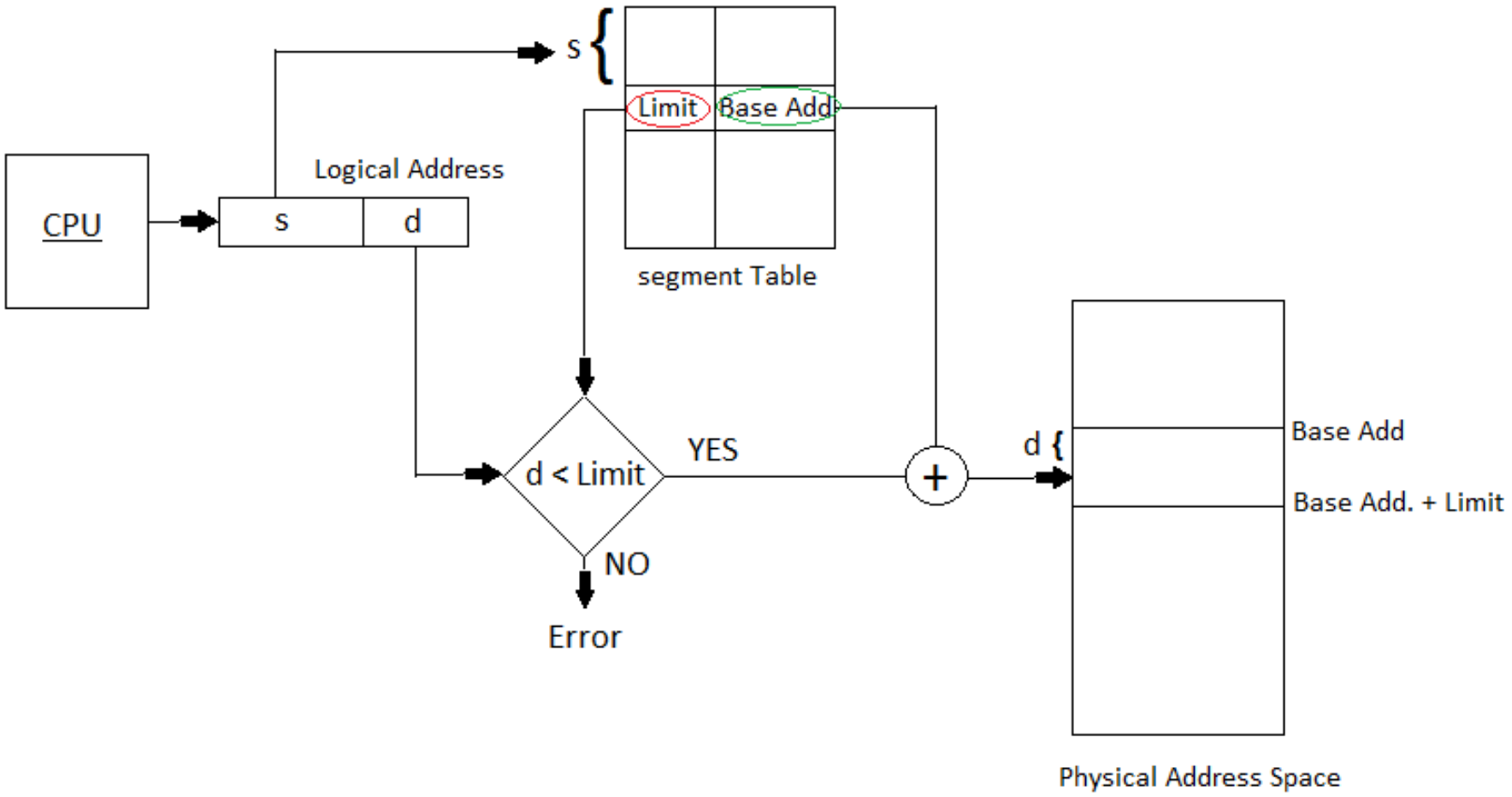
- Segment tablosundan segmentin taban (base) adresi ve uzunluğu (limit) alınır.
- Segment boyları eşit olmadığından dinamik bölmelemeye benzer özelliklere sahiptir.
- Bir kullanıcı programı birden fazla segmentden oluşabilir.
- Bir programa ilişkin segmentler bellekte ardışıl yerleşmek zorunda değildir.

Kesimli Bellek Yönetimi (Segmentation)

- Bir mantıksal adresin ofset değeri, taban ve tavan (limit) değerleri arasında olmalıdır.
- Eğer ofset değeri belirtilen sınırlar arasında değilse mantıksal adresin segmenti aşması sonucu bellek erişim hatası (segment fault) oluşur.
- Eğer ofset değeri belirtilen sınırlar arasında ise segment taban (base) adresi ile ofset toplanarak, fiziksel adres oluşturulmaktadır.

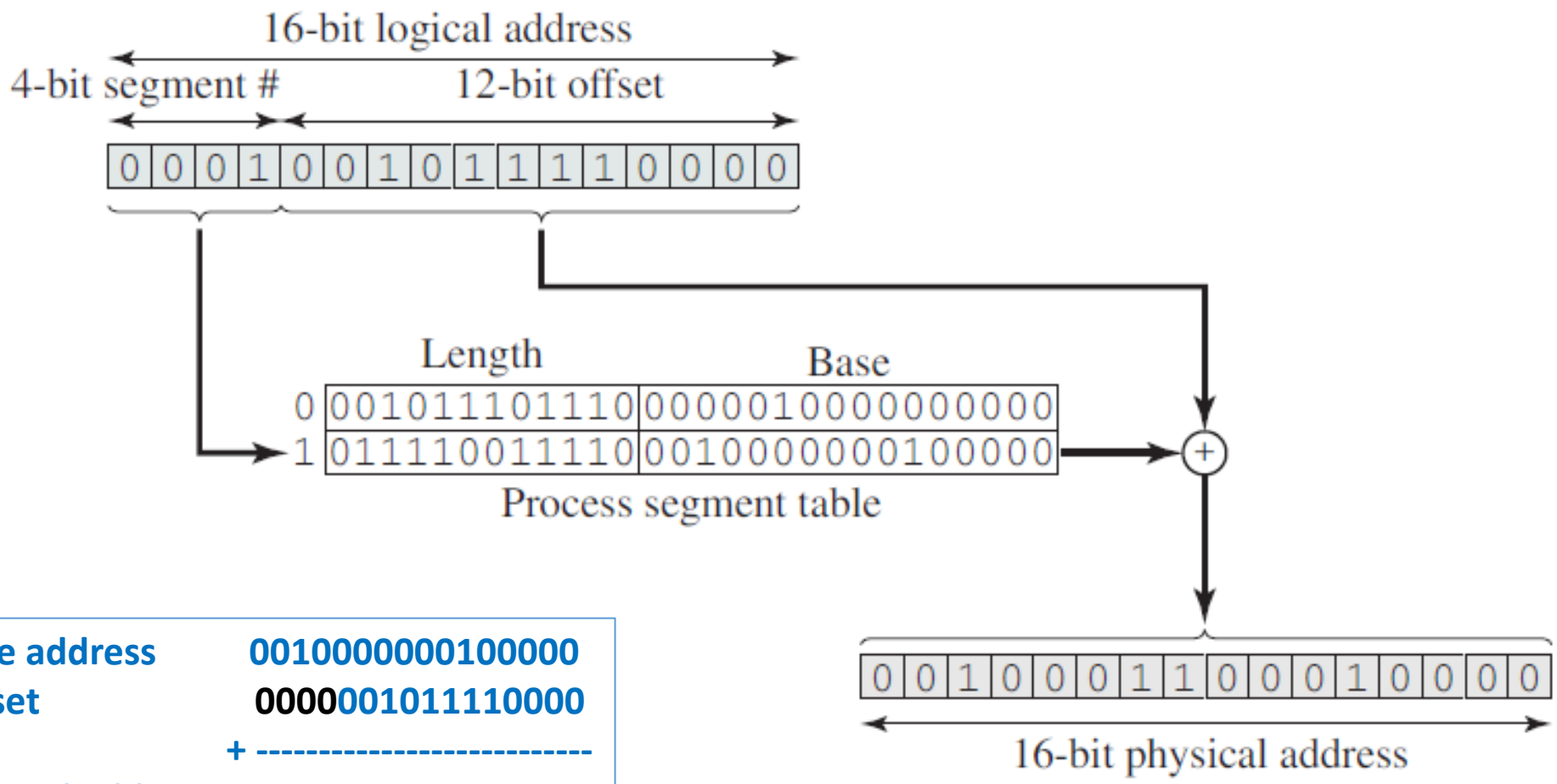
- **Fiziksel Adres (f) $\Rightarrow$ Segment Taban Adresi (s) + Ofset (d)**

Kesimli Bellek Yönetimi (Segmentation)



Kesimli Bellek Yönetimi (Segmentation)

Mantıksal Adresten Fiziksel Adres Eşleştirilmesi



Base address	**0010000000100000**
Offset	**0000001011110000**
	+ ----------
Physical address	**0010001100010000**

Kesimli Bellek Yönetimi (Segmentation)

- **Örnek**
- Segment tablosunun yandaki gibi olduğunu varsayınız.
- Aşağıdaki mantıksal adreslerin (segment no, ofset) fiziksel adres eşleştirmesini bulunuz.

a) (0, 198)

b) (2, 156)

c) (1, 530)

d) (3, 455)

e) (0, 252)

- Fiziksel adresi bulmak için aşağıdaki eşitlik kullanılabilir:
- **Fiziksel Adres (f) ⇒ Segment Taban Adresi (s) + Ofset (d)**

Segment No	Taban Adresi (base)	Uzunluk (limit)
0	660	248
1	1752	422
2	222	198
3	996	604

Kesimli Bellek Yönetimi (Segmentation)

- İlk olarak her mantıksal adresin hangi segmentde olduğu segment tablosu kullanılarak bulunmalı ve ofset değeri bu segmentin uzunluğu ile karşılaştırılmalıdır.
- Eğer ofset segmentin uzunluğundan küçükse fiziksel adres hesaplanmalı, değilse bellek erişim hatası (segment fault) oluşturulmalıdır.

Segment No	Taban Adresi (base)	Uzunluk (limit)
0	660	248
1	1752	422
2	222	198
3	996	604

	Mantıksal adres	Fiziksel adres	
a)	(0, 198)	198 < 248	660 + 198 = 858
b)	(2, 156)	156 < 198	222 + 156 = 378
c)	(1, 530)	530 $\not<$ 422	bellek erişim hatası
d)	(3, 455)	455 < 604	996 + 455 = 1451
e)	(0, 252)	252 $\not<$ 248	bellek erişim hatası

Kesimli ve sayfalı bellek yönetiminin birlikte kullanılması

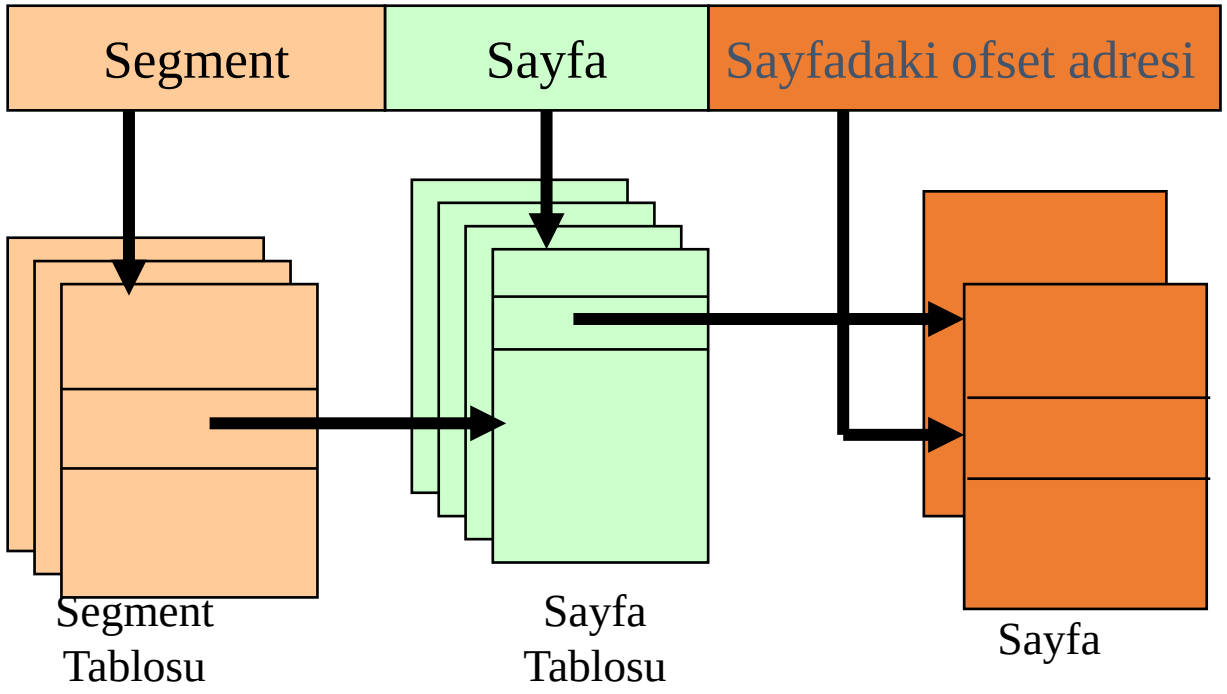
- Gerek sayfalama (paging) gerekse segmentlemenin (segmentation) kendilerine göre avantajları ve dezavantajları vardır.
- Bu iki yaklaşımı birleştirmek en verimli sonuçlardan birisi olacaktır.
- Mantıksal adres uzayı segmentlere ve segmentler de sayfalara ayrılırlar.
- Bu şekilde segmentlemede temel sakıncayı oluşturan değişik boydaki segmentleri sayfa tabanında ele almak mümkün olmaktadır.

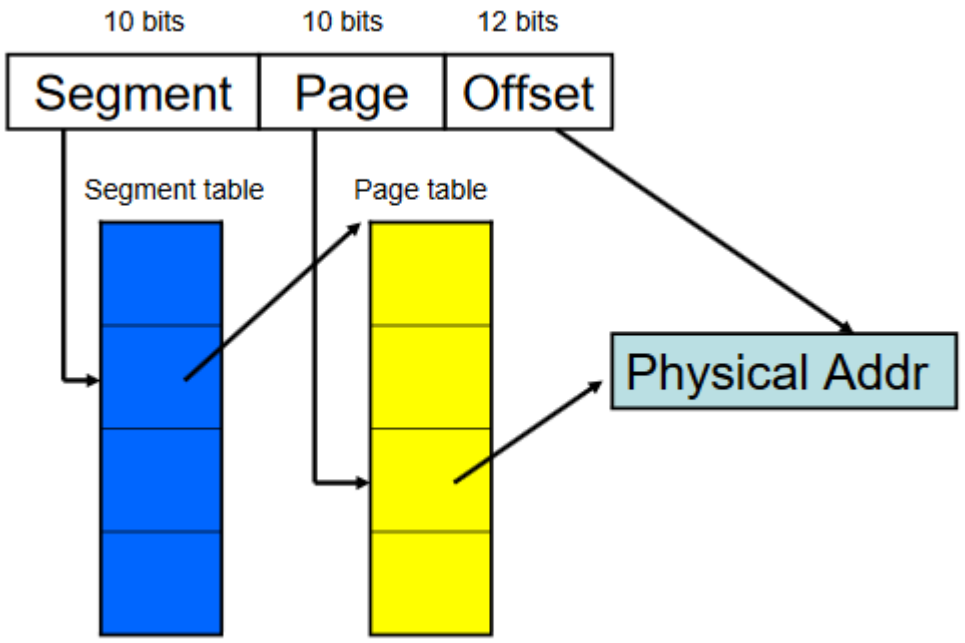
Kesimli ve sayfalı bellek yönetiminin birlikte kullanılması

- Segmentler kendi içerisinde sayfalara ayrılmaktadır.
- Mantıksal adres üç parçadan oluşmaktadır; segment, sayfa ve ofset

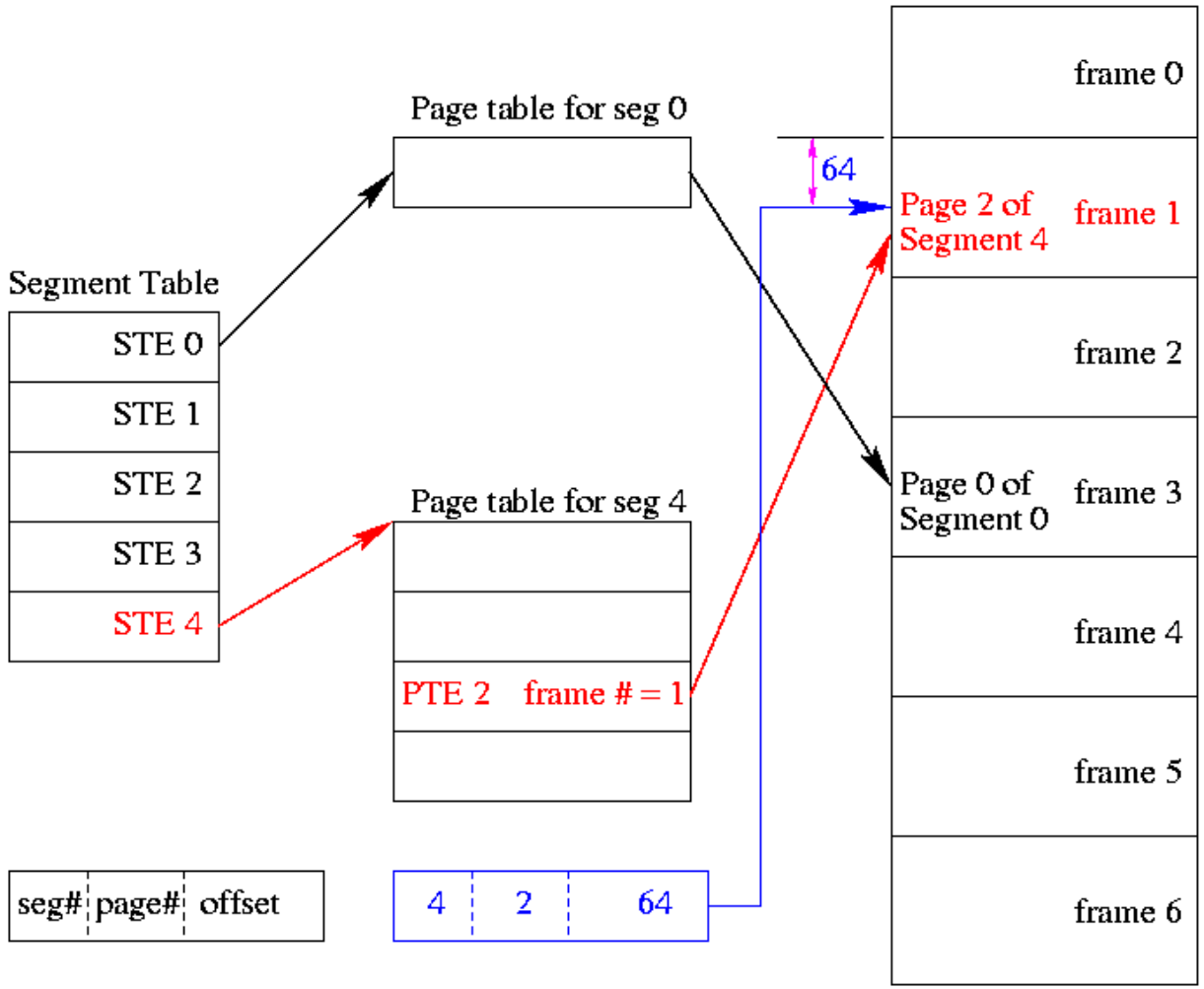
Segment numarası	Sayfa numarası	ofset

Kesimli ve sayfalı bellek yönetiminin birlikte kullanılması





Kesimli ve sayfalı bellek yönetiminin birlikte kullanılması

- Bu yöntemde, Mantıksal adreslerden fiziksel adreslere eşleştirmede iki temel tablo kullanılır:
- Segment tablosu
- Sayfa tablosu
- Segmentli-sayfalı bellek yönetiminde segment tablosu, segmentlerin başlangıç adresi yerine, bu segmentler için öngörülen ayrı ayrı sayfa tablolarının başlangıç adreslerini tutar.
- Segmentlerin uzunluk bilgisi yerine, ilgili sayfa tablosunun uzunluk bilgisine yer verilir.
- Segmentlerin belleğe yüklenmesi sayfa sayfa yapılır. Bu durumda segmentin tümünün bellekte tutulması anlamını yitirir.

Kesimli ve sayfalı bellek yönetiminin birlikte kullanılması